Dossier de rendu : GitOps & CI/CD

Nom : ELIE

Prénom : Alban

Date : Novembre 2026

Dépôt GitHub applicatif : https://github.com/FamilyTsuki/tasks-api

Dépôt GitHub de configuration (GitOps) : https://github.com/FamilyTsuki/tasks-config

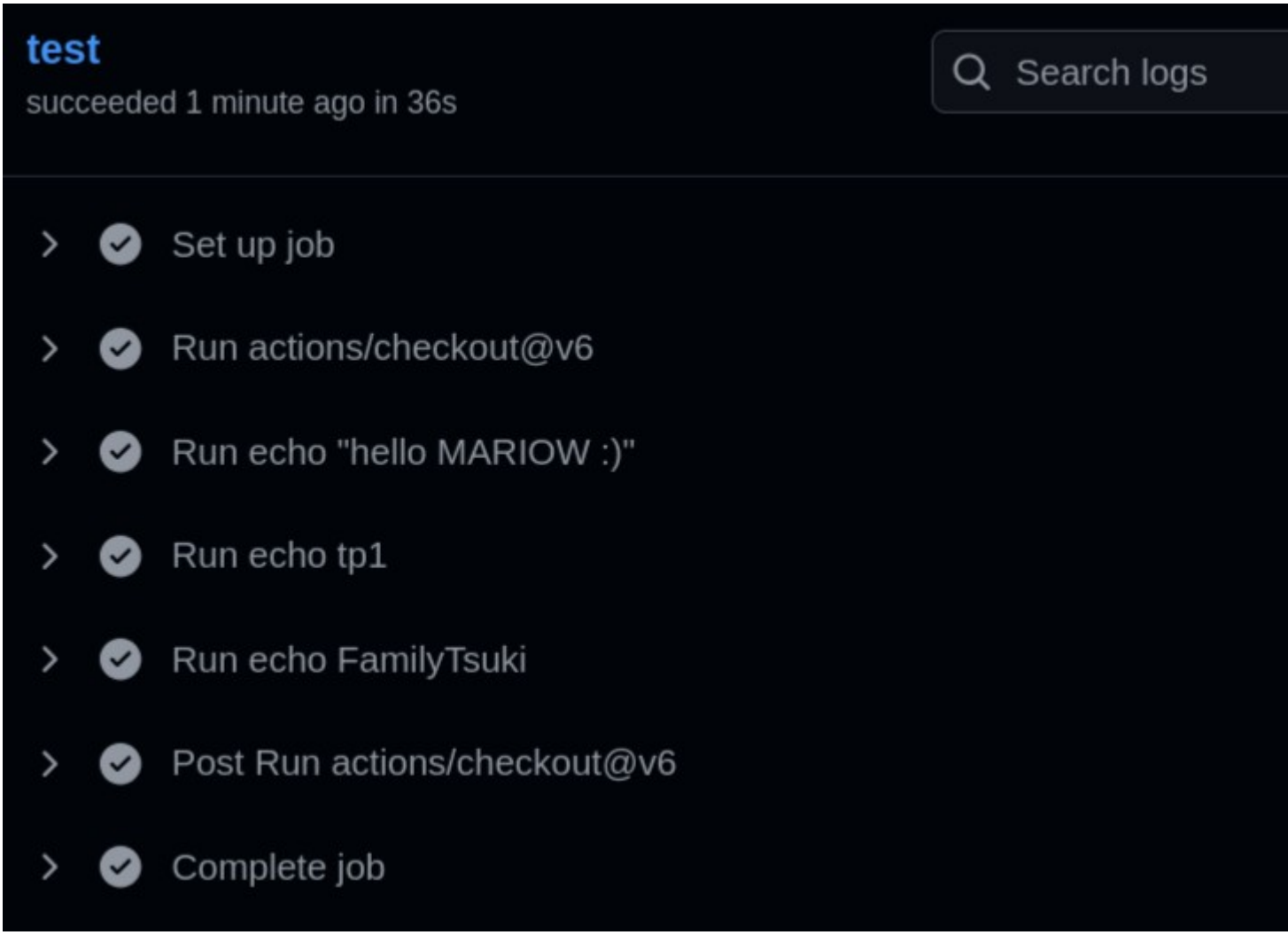
TP1 : Première CI avec GitHub Actions

Objectif

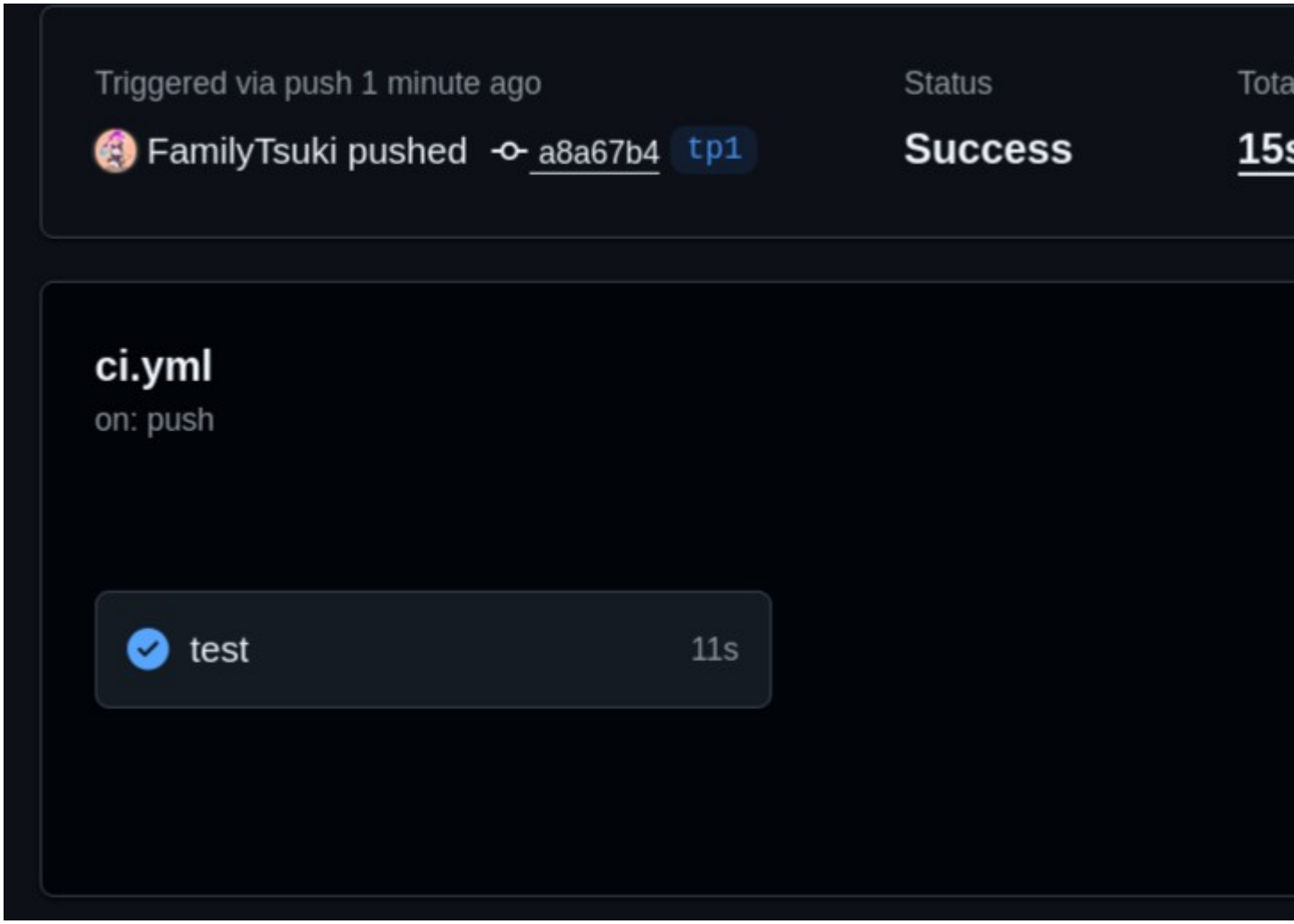
Le but de ce premier TP était de découvrir GitHub Actions en créant notre premier workflow sur les push. On a automatisé le lancement des tests pytest à chaque commit, appris à lire les logs quand un test échoue et mis en place une pull request avec badge de statut.

Captures d'écran

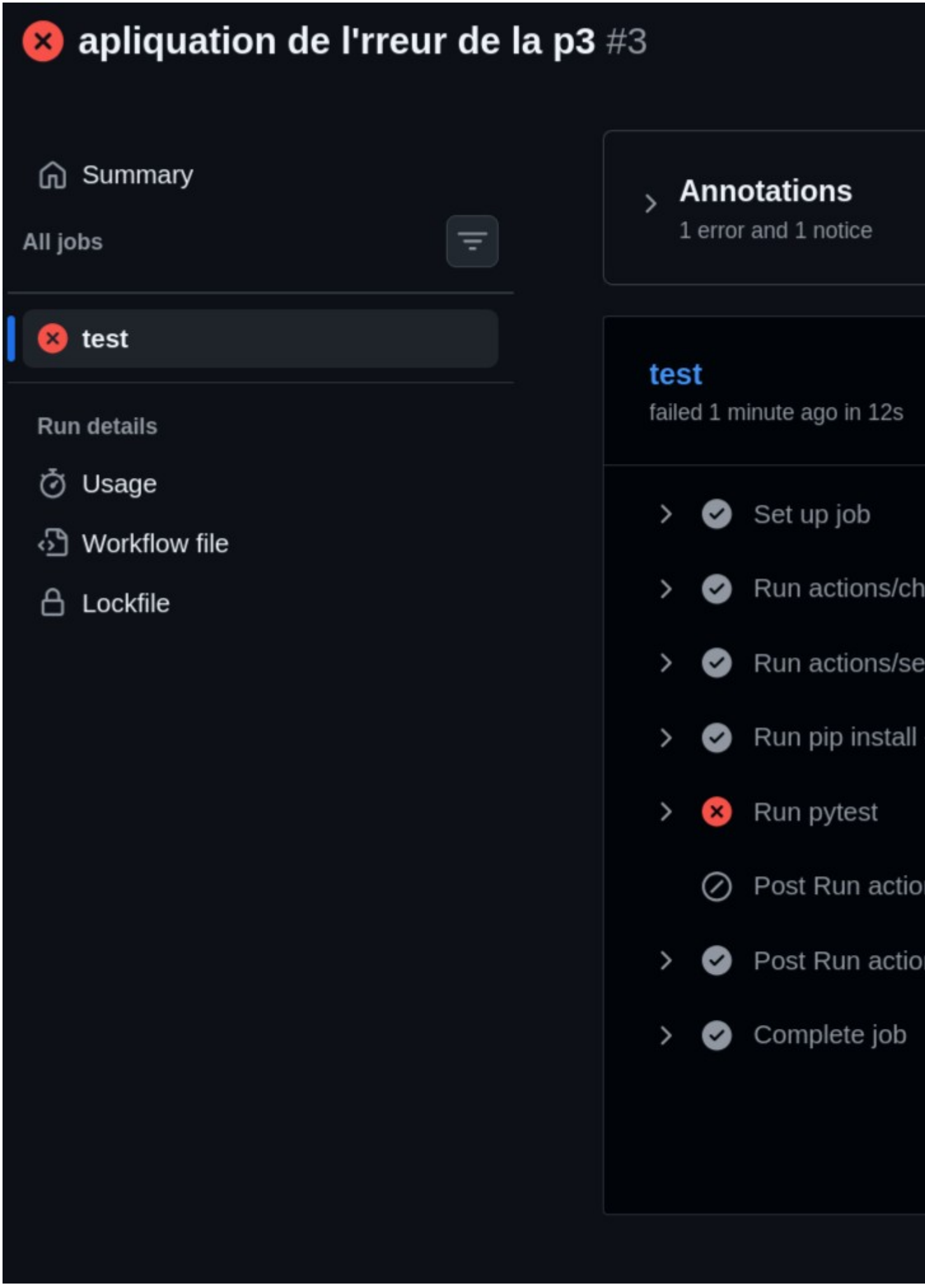
TP1-1 : Le premier workflow `Hello` réussi avec la branche et l'auteur dépliés dans les logs.



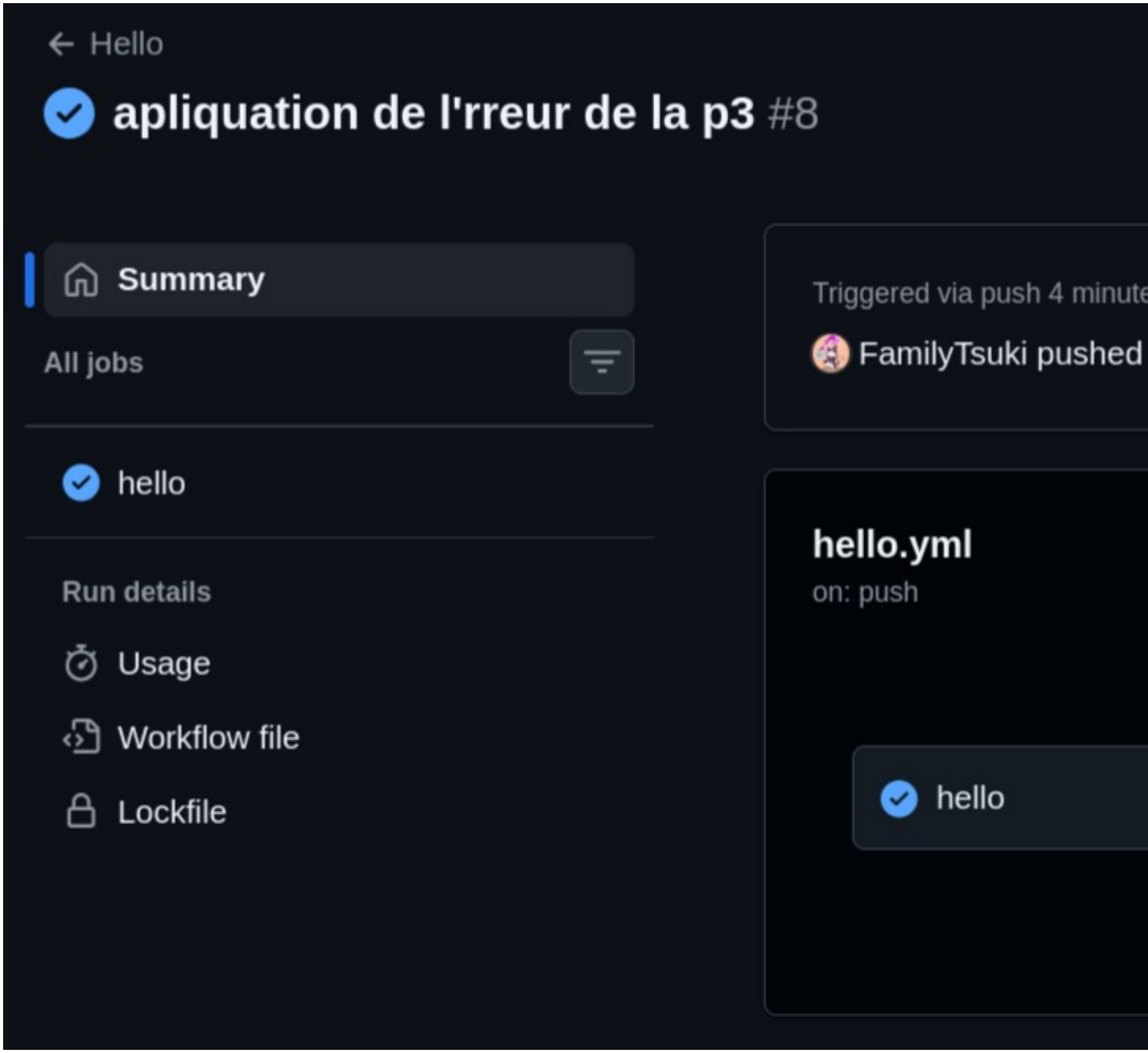
TP1-2 : Le workflow `ci.yml` réussi avec toutes les étapes d'installation et de test visibles.



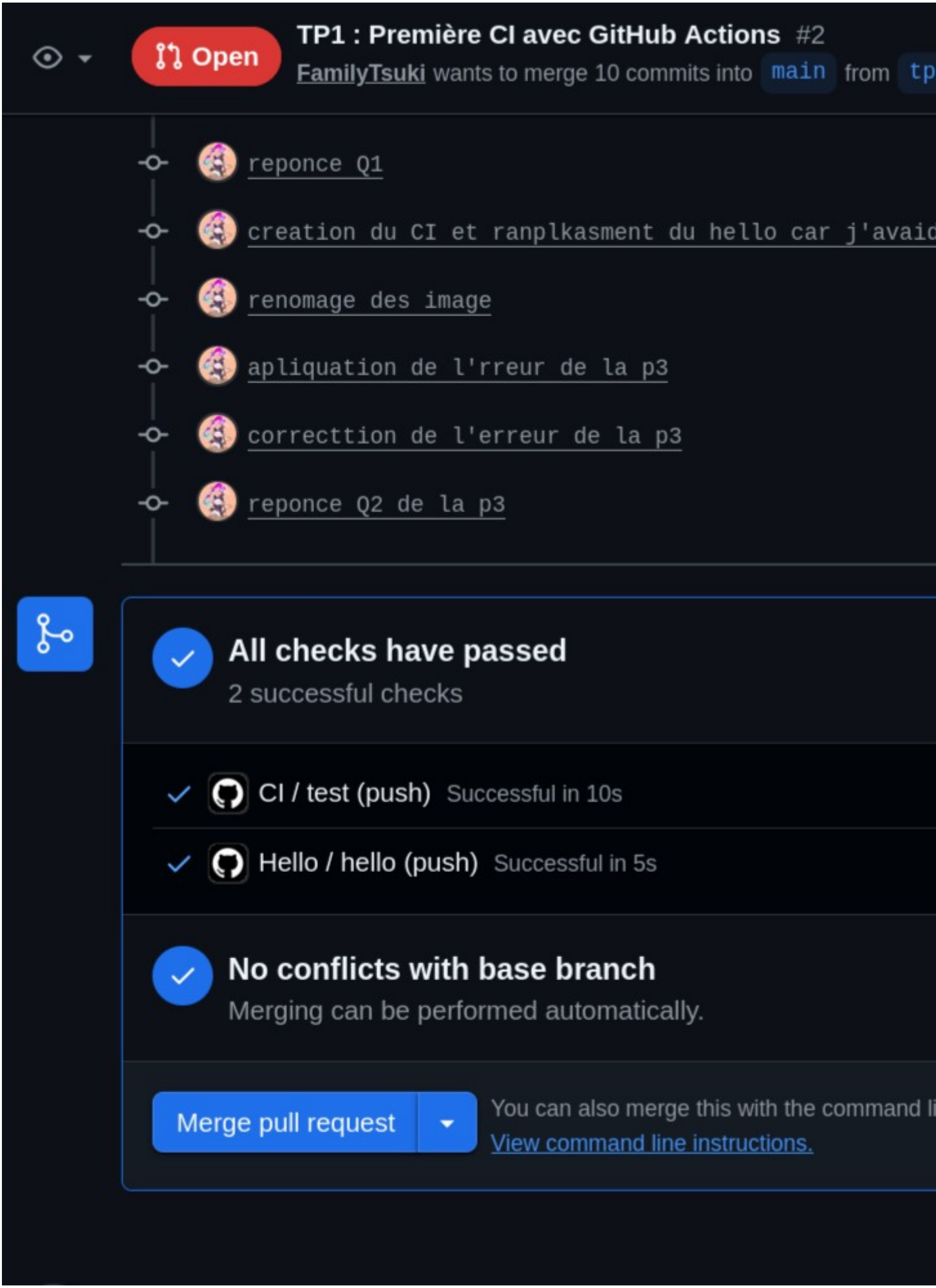
TP1-3 : Le run en échec après avoir cassé volontairement la taille max du titre dans le code.


apliquation de l'rreur de la p3 #3
Summary
All jobs
test
Run details
Usage
Workflow file
Lockfile
Annotations
1 error and 1 notice
test
failed 1 minute ago in 12s
Set up job
Run actions/ch
Run actions/se
Run pip install
Run pytest
Post Run actio
Post Run actio
Complete job

TP1-4 : Le retour au vert de la CI après correction de la valeur dans `logic.py`.



TP1-5 : La pull request ouverte avec les checks de la CI au vert avant de la fusionner.


Open
TP1 : Première CI avec GitHub Actions #2
FamilyTsuki wants to merge 10 commits into main from tp
reponce Q1
creation du CI et ranplkasment du hello car j'avaic
renomage des image
apliquation de l'rreur de la p3
correcttion de l'erreur de la p3
reponce Q2 de la p3
All checks have passed
2 successful checks
CI / test (push) Successful in 10s
Hello / hello (push) Successful in 5s
No conflicts with base branch
Merging can be performed automatically.
Merge pull request
You can also merge this with the command li
View command line instructions.

TP1-6 : La pull request fusionnée sur `main` (statut Merged violet).



TP1-7 : La page d'accueil du dépôt avec le badge vert de la CI affiché dans le README.



Réponses aux questions

Q1 : Combien de temps a duré le run « Hello » ? Où voit-on les logs de chaque étape ?

Durée totale : 39s (dont 36s pour le job). Les logs se voient sur la page du run en cliquant sur le job `test`, puis en dépliant chaque étape dans la console.

Q2 : Quel test a échoué dans la partie 3, et à quelle ligne ? Qu'est-ce qui a permis de le trouver rapidement ?

L'étape en échec était `Run pytest` : `test_is_valid_title_accepts_normal_title` à la ligne 13 de `tests/test_logic.py` et `test_add_and_list_tasks` à la ligne 29 de `tests/test_api.py` à cause de `MAX_TITLE_LENGTH = 2`. On l'a trouvé direct grâce au bloc `short test summary info` en bas des logs.

Q3 : Pourquoi attend-on que la CI soit verte avant de fusionner une pull request ?

Pour éviter de push des erreurs sur la branche `main` et être sûr que le code qu'on fusionne passe tous les tests.

Difficultés rencontrées

Au début j'ai push avec les deux variables echo séparées, du coup pour la capture TP1-1 il a fallu regrouper branche et auteur dans la même étape pour que ce soit bien lisible une fois déplié.

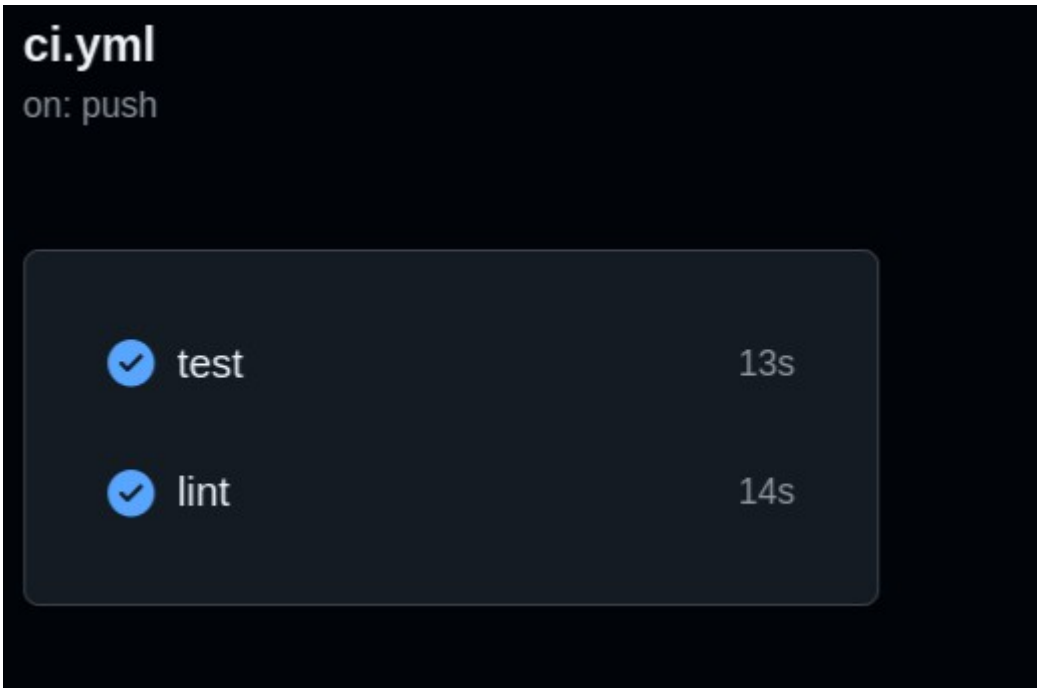
TP2 : Une CI solide

Objectif

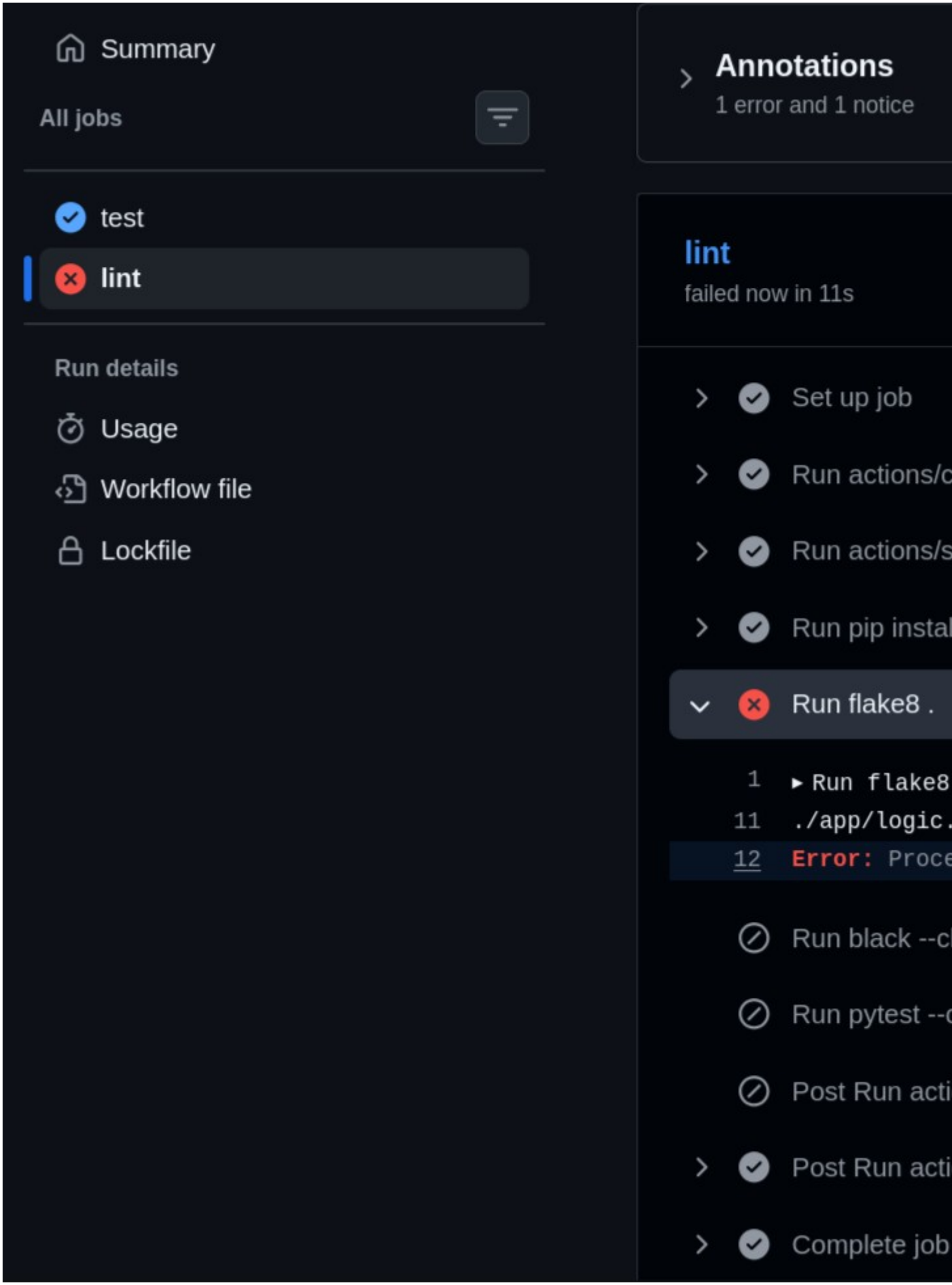
Dans ce TP on a rendu la CI plus robuste en séparant le linting du test, en ajoutant un seuil de couverture minimale et une matrice de versions Python avec du cache. On a aussi protégé la branche `main`, géré des conflits Git en équipe, masqué des secrets et réparé trois pipelines buggés.

Captures d'écran

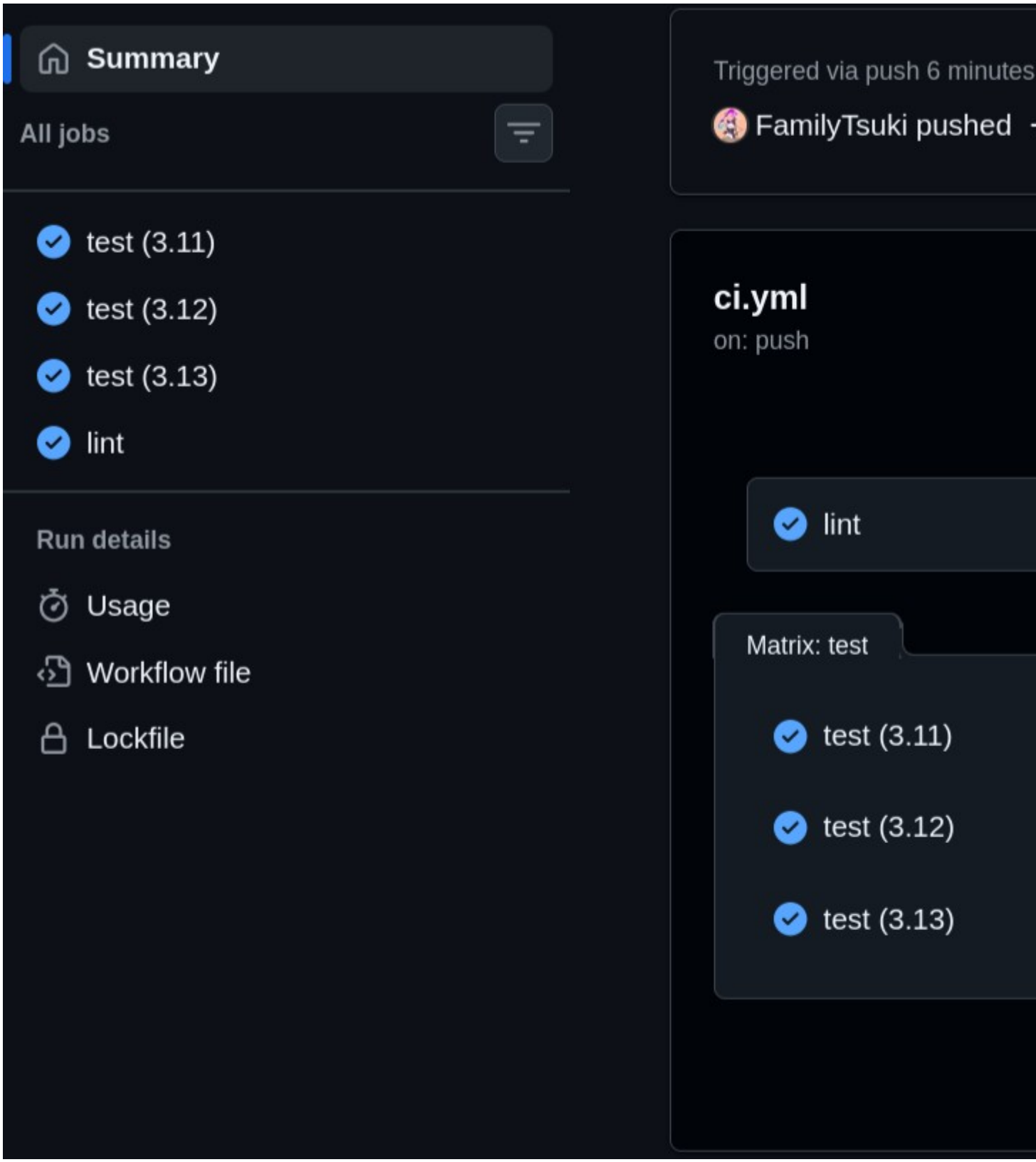
TP2-1 : Le run réussi montrant les deux jobs séparés `lint` et `test`.



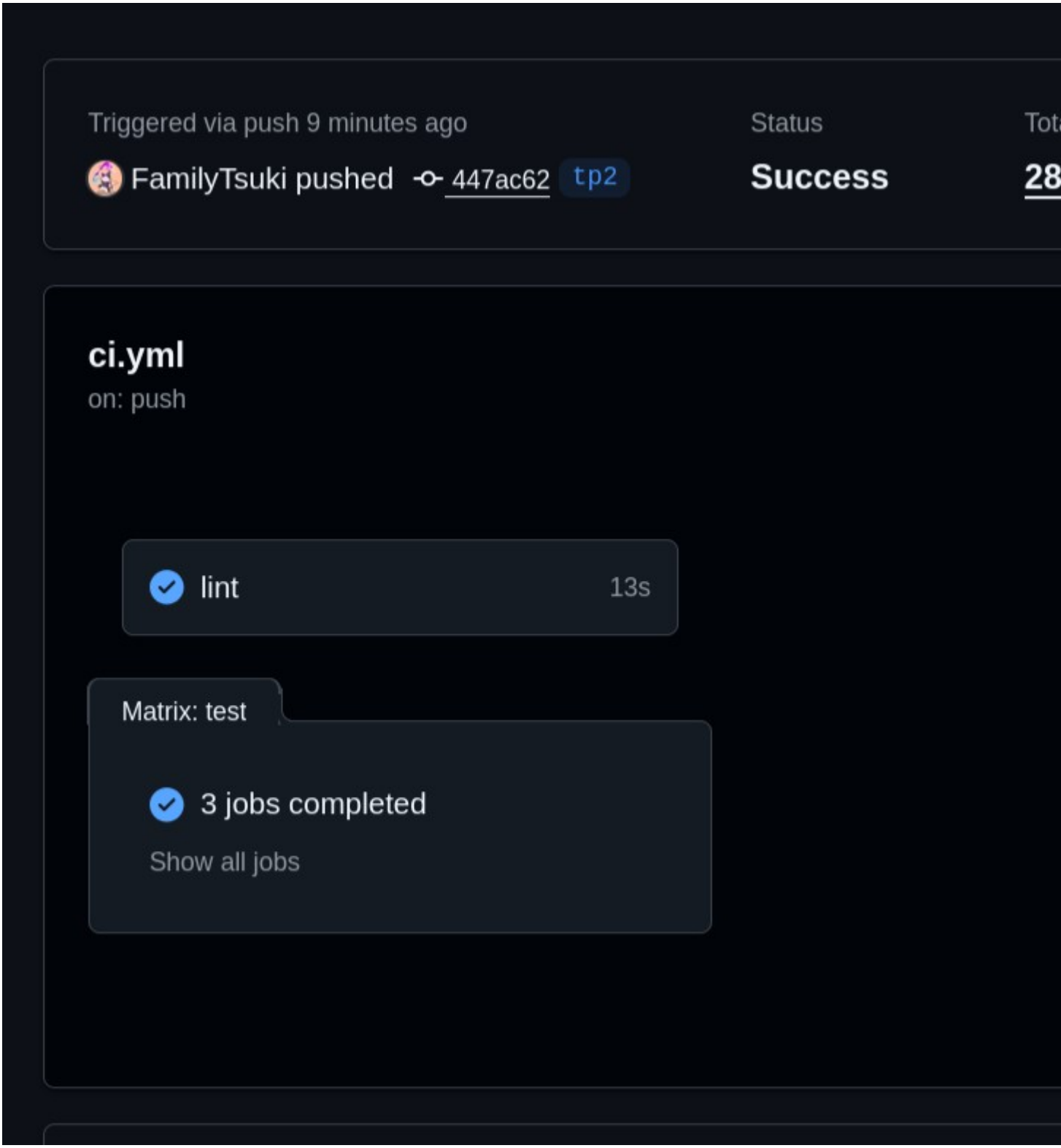
TP2-2 : Le job `lint` en rouge avec l'erreur flake8 bien visible dans les logs.

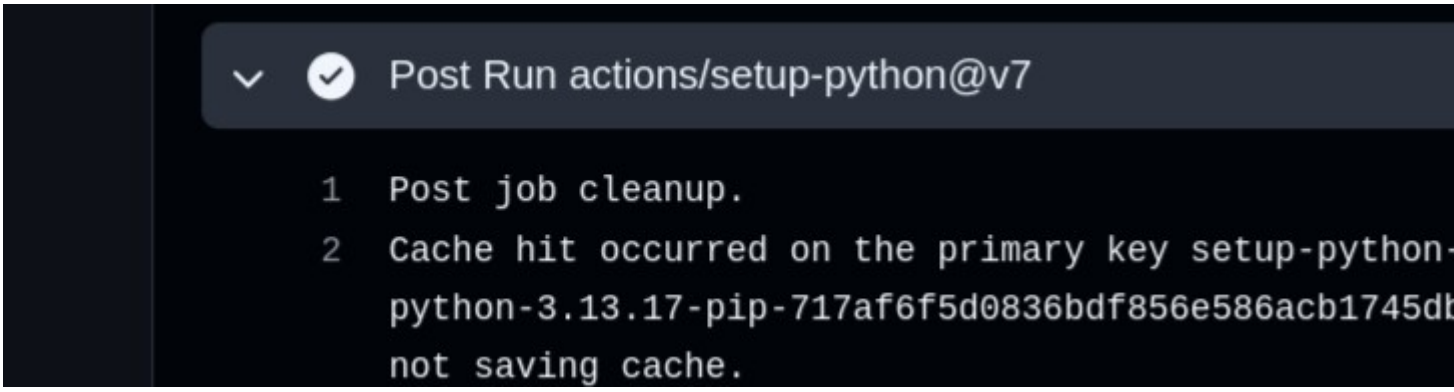

Summary
All jobs
test
lint
Run details
Usage
Workflow file
Lockfile
Annotations
1 error and 1 notice
lint
failed now in 11s
Set up job
Run actions/c
Run actions/s
Run pip instal
Run flake8 .
1 ▸ Run flake8
11 ./app/logic.
12 Error: Proce
Run black --c
Run pytest --c
Post Run acti
Post Run acti
Complete job

TP2-3 : La matrice qui fait tourner les tests en parallèle sur Python 3.11, 3.12 et 3.13.



TP2-4 : Le cache pip réutilisé au deuxième run et l'artefact `coverage.xml` téléchargeable.


Triggered via push 9 minutes ago
FamilyTsuki pushed
447ac62
tp2
Status
Success
Tot
28
ci.yml
on: push
lint
13s
Matrix: test
3 jobs completed
Show all jobs



TP2-5 : La règle de protection de `main` sur GitHub avec `lint` et `test (3.12)` obligatoires.

Branch name pattern *

main

Applies to 1 branch

`main`

Protect matching branches

- [x] **Require a pull request before merging**
  When enabled, all commits must be made to a non-protected branch and submitted v
  into a branch that matches this rule.

  - [ ] **Require approvals**
    When enabled, pull requests targeting a matching branch require a number of app
    they can be merged.

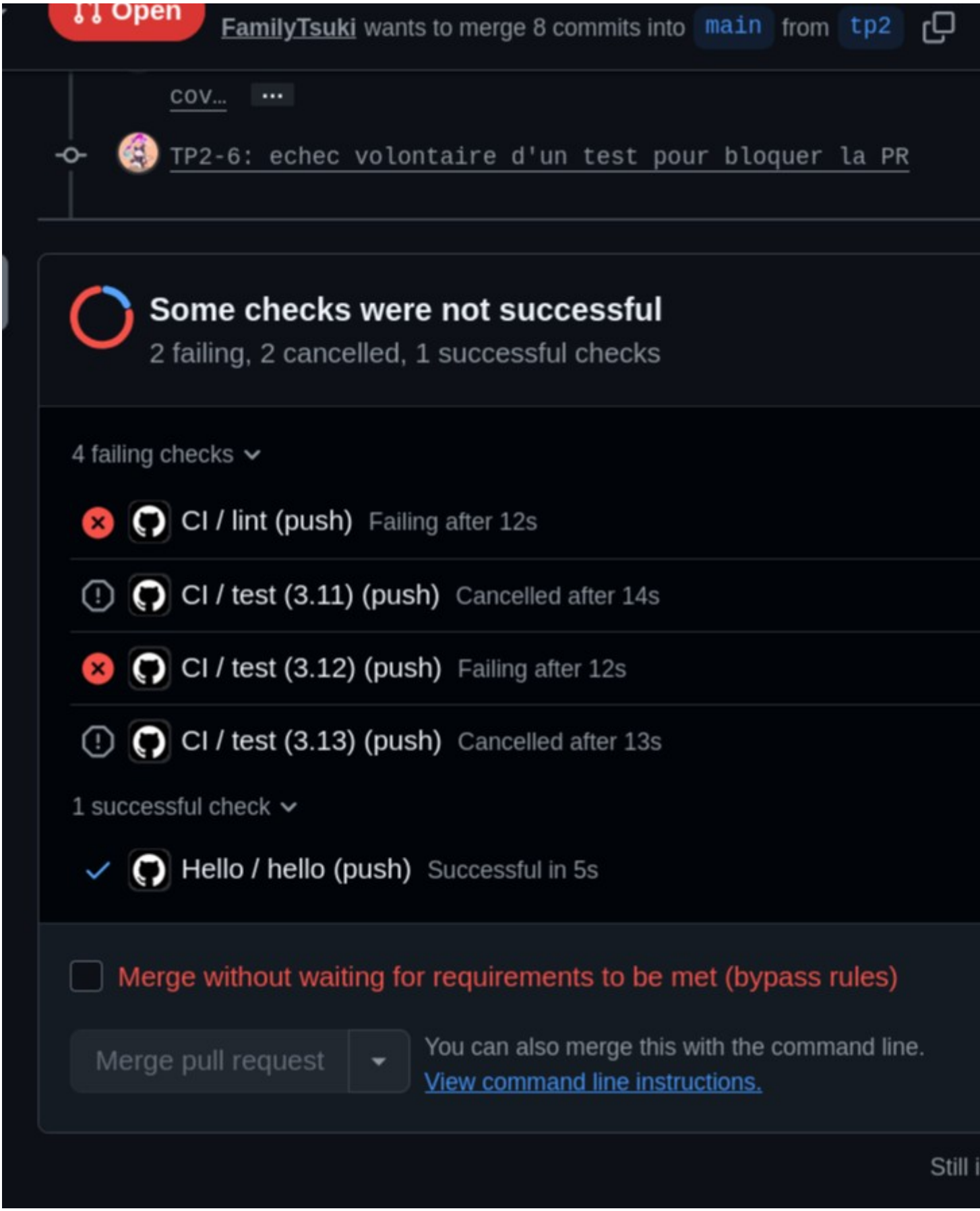
  - [ ] **Dismiss stale pull request approvals when new commits are push**
    New reviewable commits pushed to a matching branch will dismiss pull request re

  - [ ] **Require review from Code Owners**
    Require an approved review in pull requests including files with a designated code

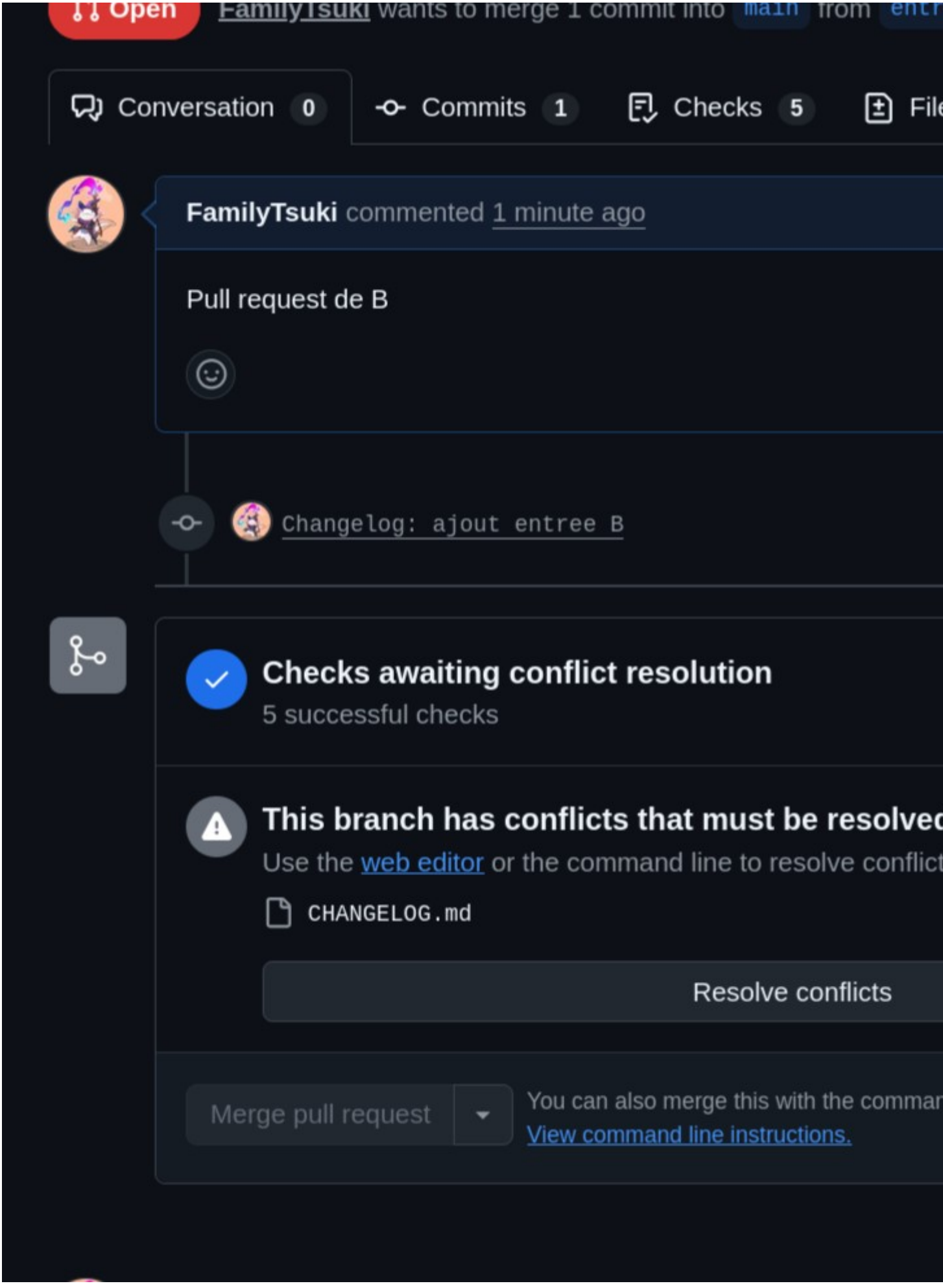
  - [ ] **Require approval of the most recent reviewable push**
    Whether the most recent reviewable push must be approved by someone other th

- [x] **Require status checks to pass before merging**
  Choose which status checks must pass before branches can be merged into a branch
  commits must first be pushed to another branch, then merged or pushed directly to a
  checks have passed.

TP2-6 : La pull request bloquée car un test obligatoire a échoué (bouton gris/rouge).



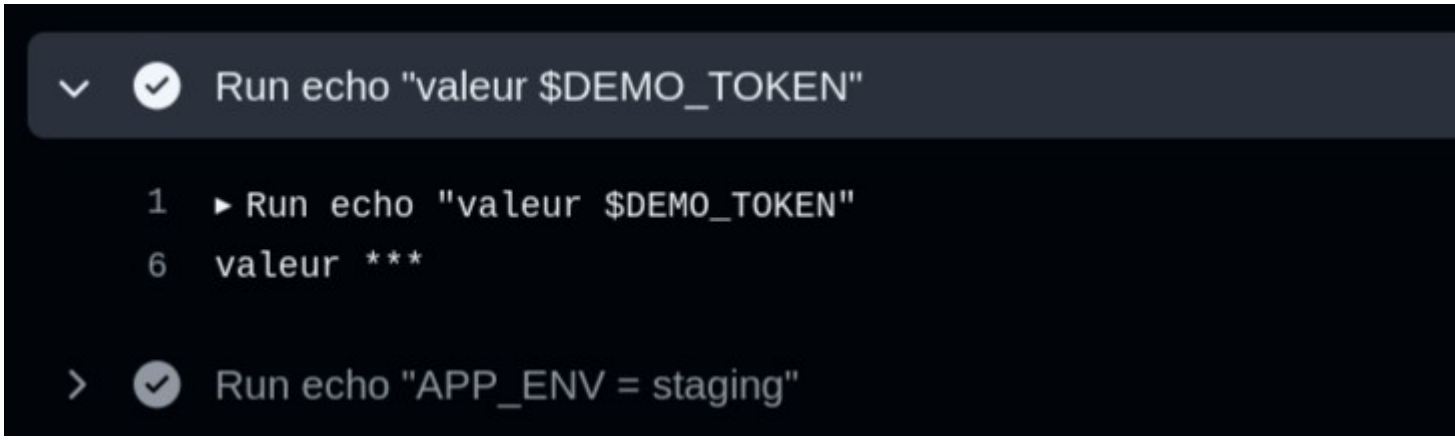
TP2-7 : La pull request de B qui affiche le conflit avec la branche `main`.


Open
FamilyTsuki wants to merge 1 commit into main from entr
Conversation 0
Commits 1
Checks 5
File
FamilyTsuki commented 1 minute ago
Pull request de B
Changelog: ajout entree B
Checks awaiting conflict resolution
5 successful checks
This branch has conflicts that must be resolved
Use the web editor or the command line to resolve conflict
CHANGELOG.md
Resolve conflicts
Merge pull request
You can also merge this with the comman
View command line instructions.

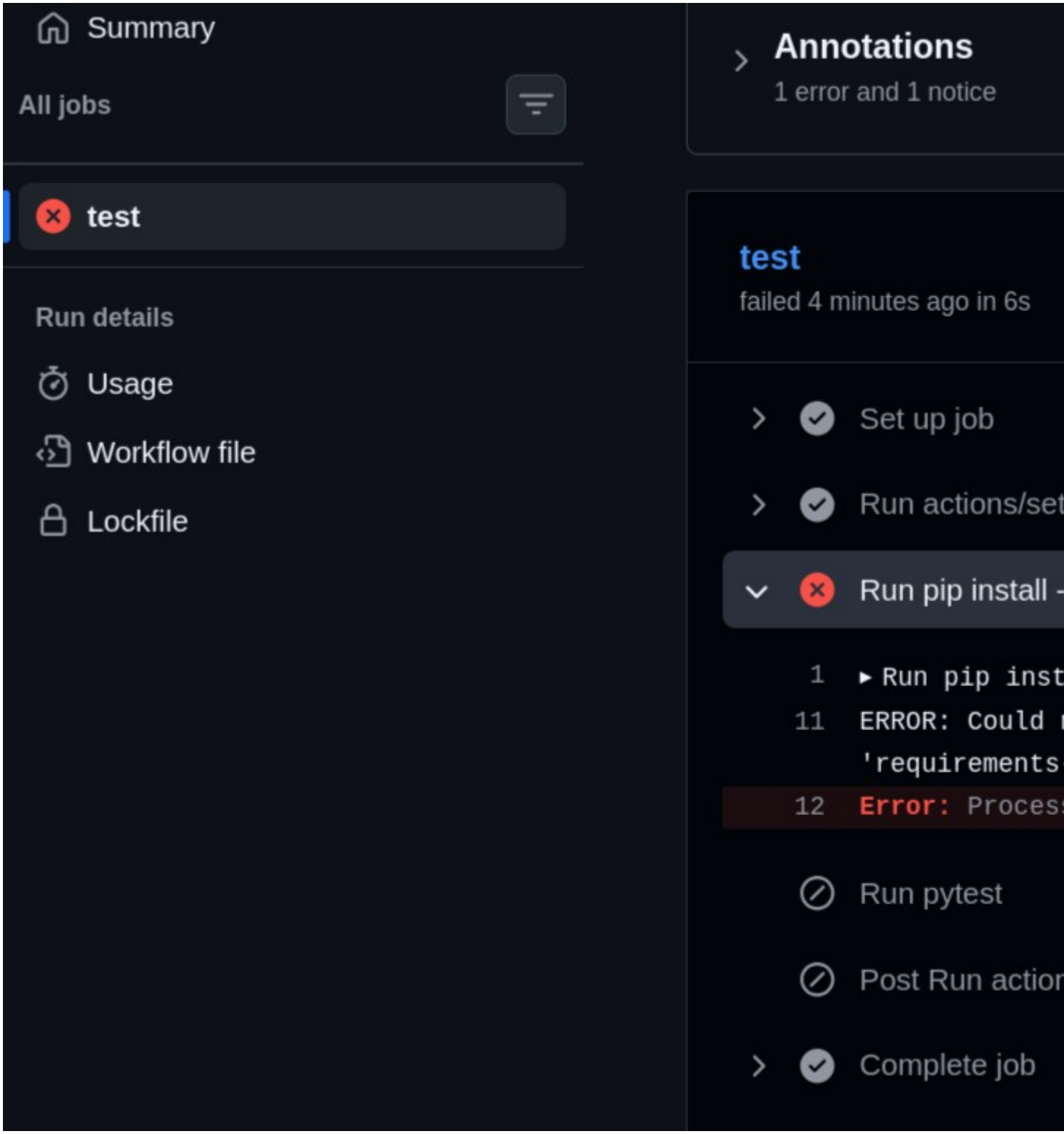
TP2-8 : Le fichier `CHANGELOG.md` sur GitHub contenant bien les deux lignes après fusion.

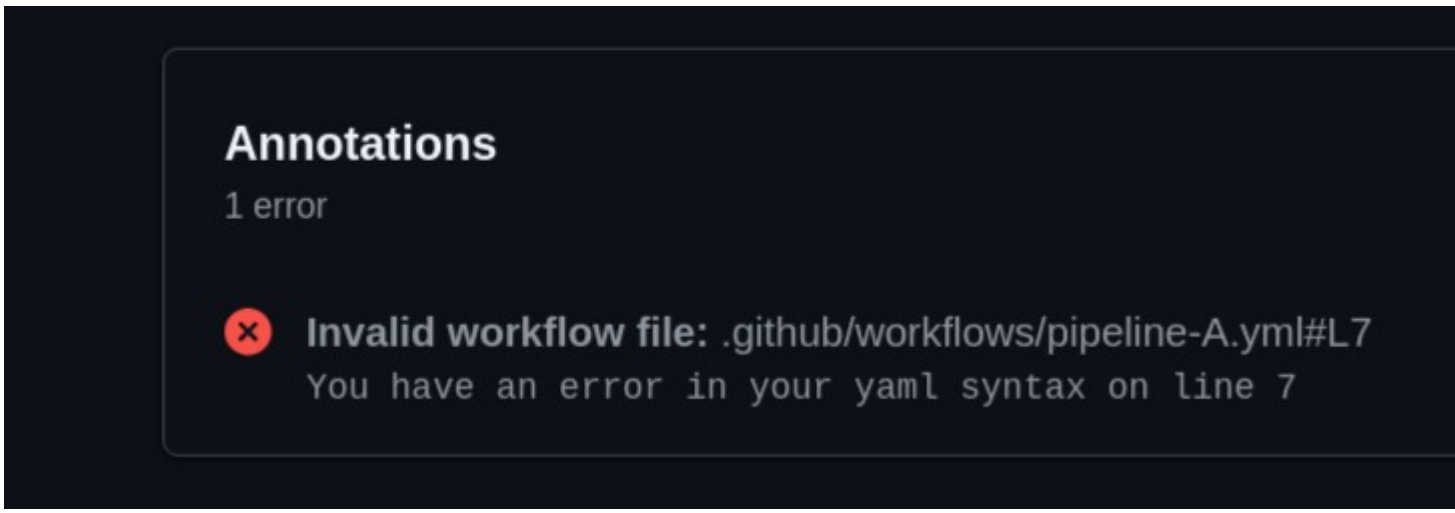


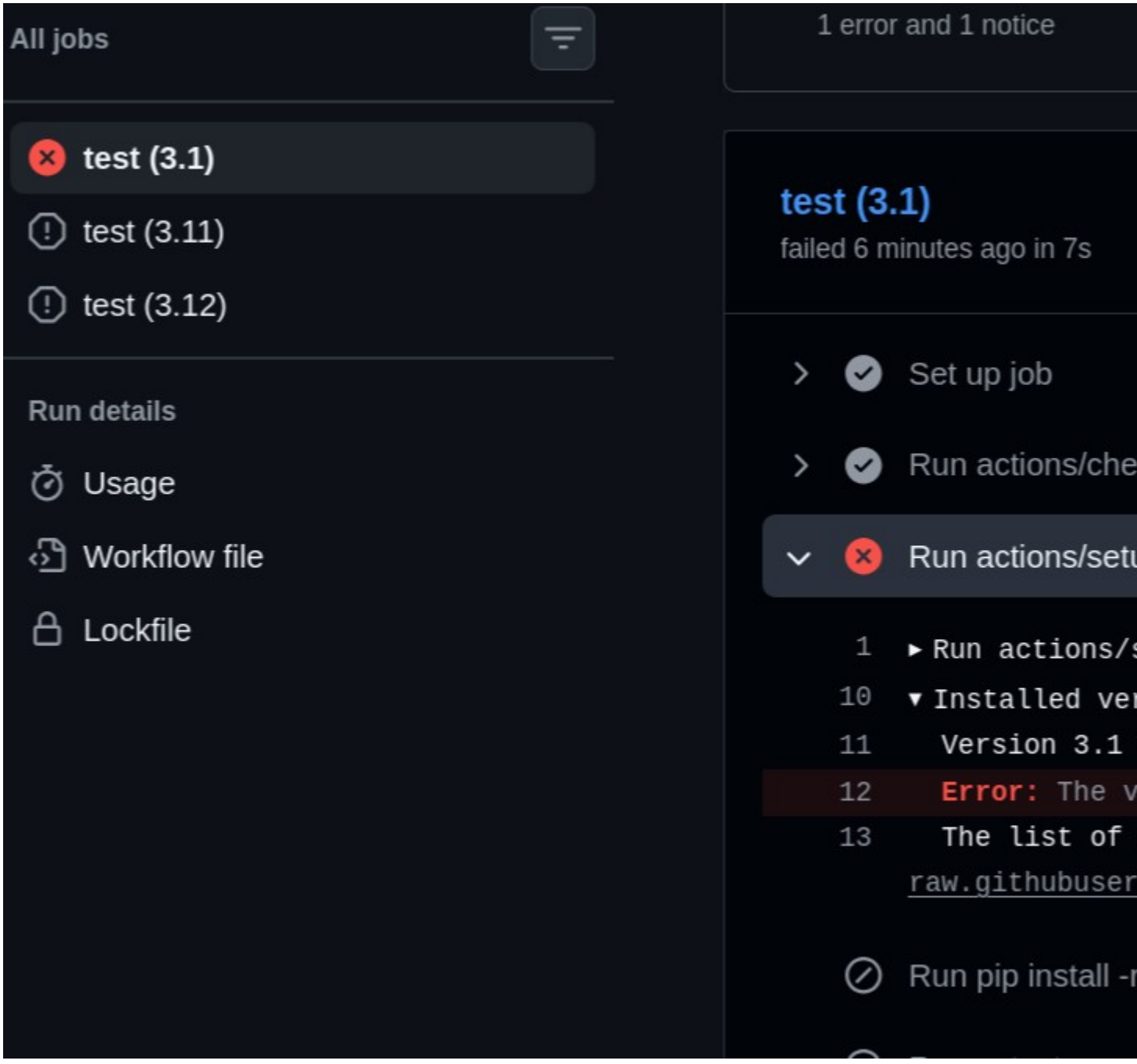
TP2-9 : Les logs de GitHub Actions où la valeur secrète est masquée par `***`.



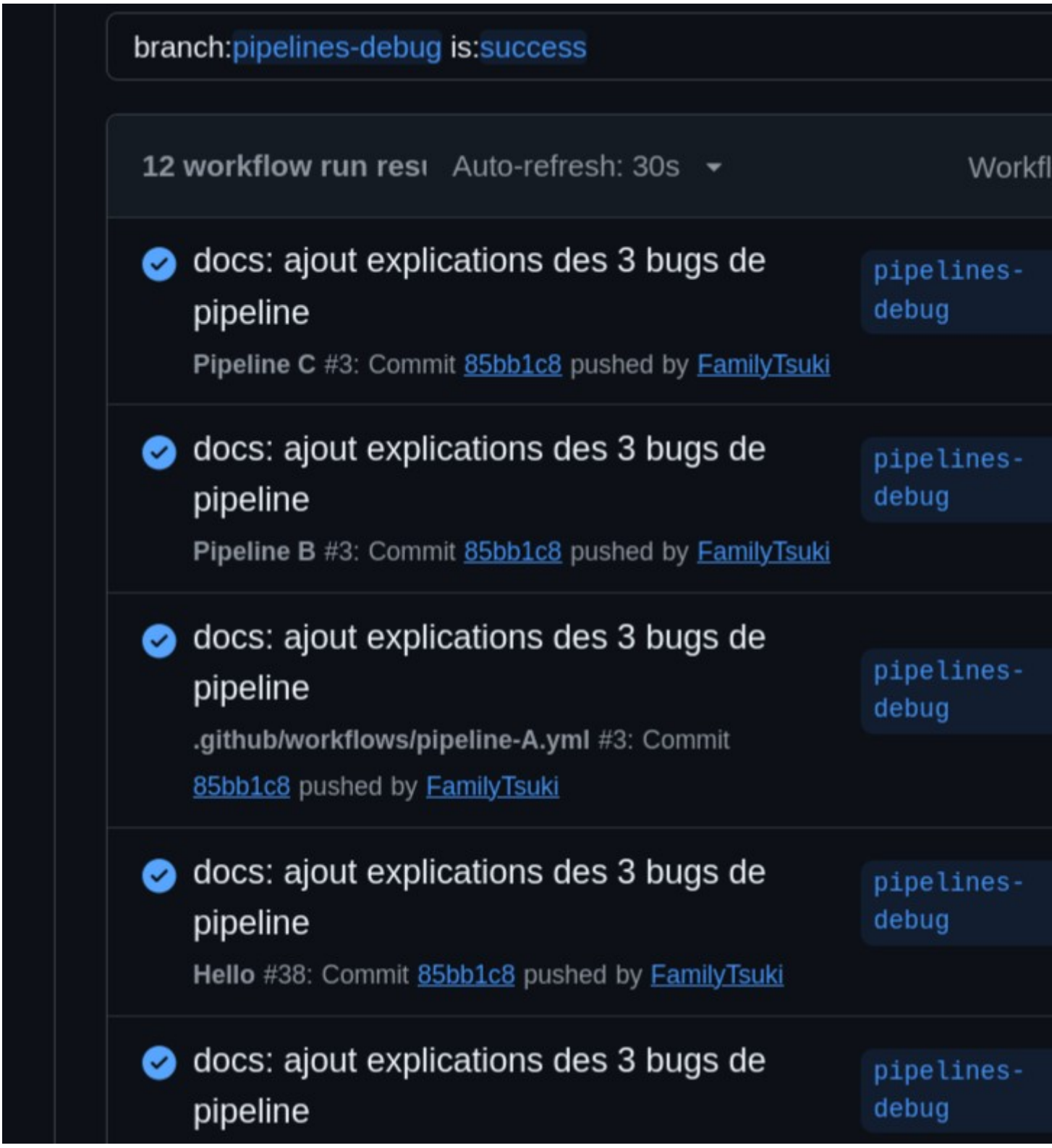
TP2-10 : Les 3 pipelines d'exercice en échec (A : syntaxe yaml, B : checkout manquant, C : python 3.10 lu en 3.1).




Annotations
1 error
Invalid workflow file: .github/workflows/pipeline-A.yml#L7
You have an error in your yaml syntax on line 7


All jobs
test (3.1)
test (3.11)
test (3.12)
Run details
Usage
Workflow file
Lockfile
1 error and 1 notice
test (3.1)
failed 6 minutes ago in 7s
Set up job
Run actions/che
Run actions/setu
1 ▸ Run actions/
10 ▾ Installed ver
11 Version 3.1
12 Error: The v
13 The list of
raw.githubuser
Run pip install -r

TP2-11 : La liste des runs montrant les 3 pipelines corrigés au vert.



Réponses aux questions

Q1 : Quelle différence entre un linter (flake8) et un formateur (black) ? Pourquoi séparer `lint` et `test` en deux jobs ?

Flake cherche les erreurs de style et les bugs (variables inutilisées, etc.) alors que Black réécrit le

code pour lui donner un style uniforme. On les sépare en deux jobs pour voir simplement à quelle étape est l'erreur et pouvoir les faire tourner en même temps.

Q2 : Pourquoi tester sur plusieurs versions de Python ? Que gagne-t-on avec le cache ? Un artefact, c'est quoi ?

Pour être sûr que le code marche sur toutes les versions que les utilisateurs peuvent avoir et voir direct si une version casse quelque chose. Avec le cache on gagne du temps parce que pip réutilise les paquets déjà téléchargés au lieu de tout re-télécharger. Un artefact c'est un fichier produit par le run (ici `coverage.xml`) qu'on peut récupérer après.

Q3 : Pourquoi interdire de pousser directement sur `main` ?

Pour interdire de casser la branche principale avec du code pas testé, ça force à passer par une PR et à attendre que la CI soit verte.

Q4 : Pourquoi Git ne peut-il pas résoudre ce conflit seul ? Qu'est-ce qui a vérifié le code après la résolution ?

Parce que les deux branches ont modifié la même ligne du CHANGELOG et Git ne sait pas laquelle garder ou s'il faut garder les deux, c'est à nous de choisir. C'est la CI qui s'est relancée sur la PR après le commit de résolution et a refait passer lint + tests.

Q5 : Pourquoi masquer la valeur dans les logs ? Qu'est-ce qui se passe si on commite un secret dans Git ? Différence secret vs variable ?

Pour que personne ne puisse le lire dans les logs vu qu'ils sont visibles par tous sur un dépôt public. Si on commite un secret, il reste dans l'historique Git pour toujours même si on le supprime après, donc il faut le considérer comme compromis et le révoquer direct. On utilise un secret pour ce qui est sensible (tokens, mdp) et une variable pour la config non secrète.

Q6 : Analyse des 3 pipelines réparés :

- *Pipeline A :* erreur d'indentation yaml à la ligne 12 (`- run` décalé de 7 espaces au lieu de 6). GitHub Actions a refusé de lancer le job. Pour éviter ça : mettre un linter yaml dans son éditeur.

- *Pipeline B :* il manquait l'étape `actions/checkout`. Le `pip install` a planté avec `FileNotFoundError` sur `requirements-dev.txt` car le runner démarre vide. Pour éviter ça : toujours mettre checkout au début.

- *Pipeline C :* les versions python dans la matrix étaient sans guillemets `[3.10, 3.11, 3.12]`, du coup `3.10` a été lu comme `3.1`. L'étape `setup-python` a cherché Python 3.1 qui n'existe pas. Pour éviter ça : toujours mettre des guillemets autour des versions.

Difficultés rencontrées

Pour le conflit, il a fallu faire attention à bien retirer les balises `<<<<<<<` et `>>>>>>>` dans le CHANGELOG et garder les deux phrases. Pour le pipeline C le bug était vicieux parce que c'est le parseur YAML qui transforme `3.10` en nombre flottant `3.1`.

TP3 : Docker et livraison continue

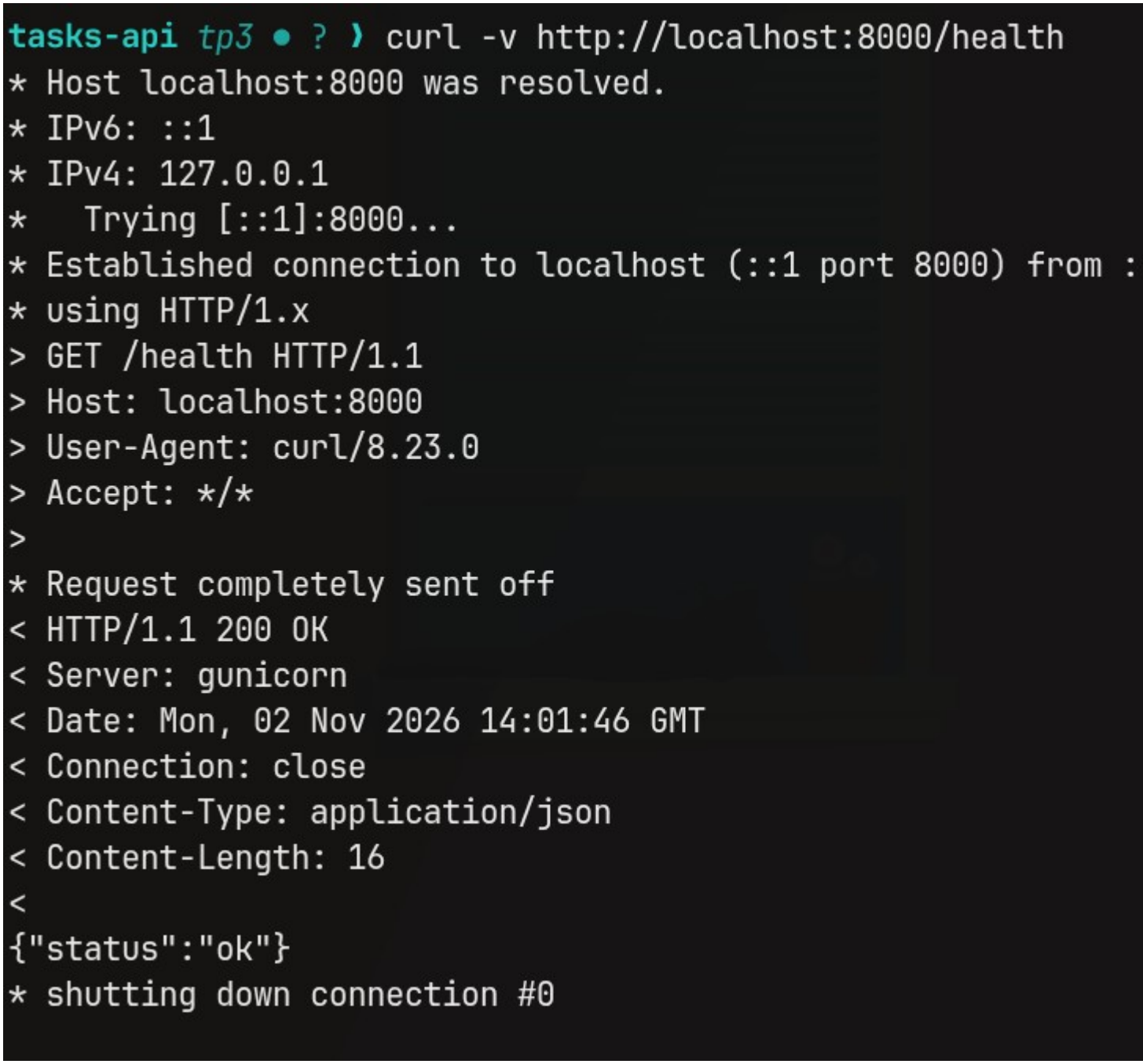
Objectif

L'objectif du TP3 était de conteneuriser notre API avec un Dockerfile léger et sécurisé (utilisateur non-root), d'utiliser Docker Compose en local, puis de créer un workflow qui teste, construit et publie automatiquement les images versionnées sur GitHub Packages (GHCR) avec validation manuelle pour la prod.

Captures d'écran

TP3-1 : La construction de l'image Docker locale et le `curl /health` qui répond `{"status":"ok"}`.

```
tasks-api tp3 × sudo docker build -t tasks-api:local .
[+] Building 14.2s (11/11) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 1.18kB
 => [internal] load metadata for docker.io/library/python:
 => [internal] load .dockerignore
 => => transferring context: 181B
 => [1/6] FROM docker.io/library/python:3.12-slim@sha256:b
 => => resolve docker.io/library/python:3.12-slim@sha256:b
 => => sha256:acca7542dfc736e9ce20cbefc58b2d09822276d15d4a
 => => sha256:f574edac59d9f092df596f54e5b52656c803fd3ae308
 => => sha256:3eb7711515067656 8f88e2957181b8f7e3014e797f1c
 => => sha256:6fc242d221a673263eabd14a9ad9584d3177c96e5cb5
 => => extracting sha256:6fc242d221a673263eabd14a9ad9584d3
 => => extracting sha256:3eb7711515067656 8f88e2957181b8f7e
 => => extracting sha256:f574edac59d9f092df596f54e5b52656c
 => => extracting sha256:acca7542dfc736e9ce20cbefc58b2d098
 => [internal] load build context
 => => transferring context: 1.70MB
 => [2/6] WORKDIR /app
 => [3/6] COPY requirements.txt .
 => [4/6] RUN pip install --no-cache-dir -r requirements.t
 => [5/6] COPY . .
 => [6/6] RUN useradd --create-home --shell /bin/bash app
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:2ed9b5142fb4f872249a207f4
 => => exporting config sha256:1a4539a027d3c4b18cd4a47d711
 => => exporting attestation manifest sha256:ef06a336a3927
 => => exporting manifest list sha256:fd4e5c6874ca1c23e506
 => => naming to docker.io/library/tasks-api:local
 => => unpacking to docker.io/library/tasks-api:local
```



TP3-2 : Lancement avec `docker compose up` et test du `/version` qui répond `local`.

```
[+] Building 0.8s (13/13) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 550B
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 1.28kB
 => [internal] load metadata for docker.io/library/python:3.
 => [internal] load .dockerignore
 => => transferring context: 181B
 => [internal] load build context
 => => transferring context: 2.13kB
 => [1/6] FROM docker.io/library/python:3.12-slim@sha256:b18
 => => resolve docker.io/library/python:3.12-slim@sha256:b18
 => CACHED [2/6] WORKDIR /app
 => CACHED [3/6] COPY requirements.txt .
 => CACHED [4/6] RUN pip install --no-cache-dir -r requireme
 => CACHED [5/6] COPY . .
 => CACHED [6/6] RUN useradd --create-home --shell /bin/bash
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:cb766a2d11bbfe055217ce08713
 => => exporting config sha256:12bb90f5b46201f4c5f310065d50e
 => => exporting attestation manifest sha256:365e3fb86a1fc49
 => => exporting manifest list sha256:e680c6ddd1d444ebe1fed5
 => => naming to docker.io/library/tasks-api-api:latest
 => => unpacking to docker.io/library/tasks-api-api:latest
 => resolving provenance for metadata file
[+] up 2/2
 ✔Image tasks-api-api        Built
 ✔Container tasks-api-api-1 Recreated
Attaching to api-1
api-1  | [2026-11-02 14:15:35 +0000] [1] [INFO] Starting gun
api-1  | [2026-11-02 14:15:35 +0000] [1] [INFO] Listening at
```



TP3-3 : Le workflow GitHub Actions montrant le job `test` suivi du job `build` au vert.

Triggered via push 3 minutes ago

FamilyTsuki pushed 48d43b9 tp3

Status

Success

Total d

1m 1

docker.yml

on: push



build summary

Docker Build summary

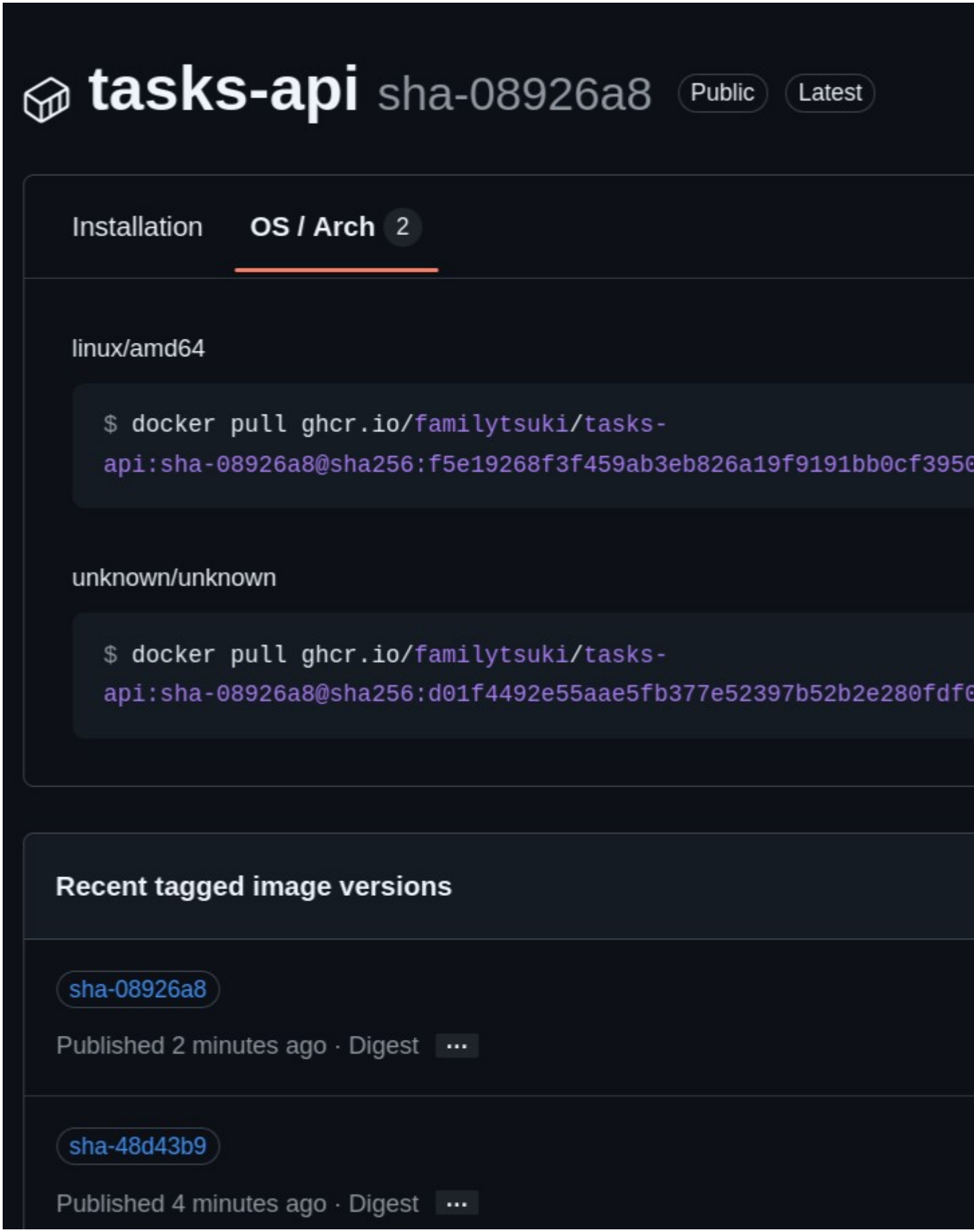
For a detailed look at the build, download the following build record
Docker Desktop's Builds view.
Build records include details such as timing, dependencies, results,
information about a build. Learn more

⬇️ **FamilyTsuki~tasks-api~YT7JQI.dockerbuild** (42.82 KB - inclu

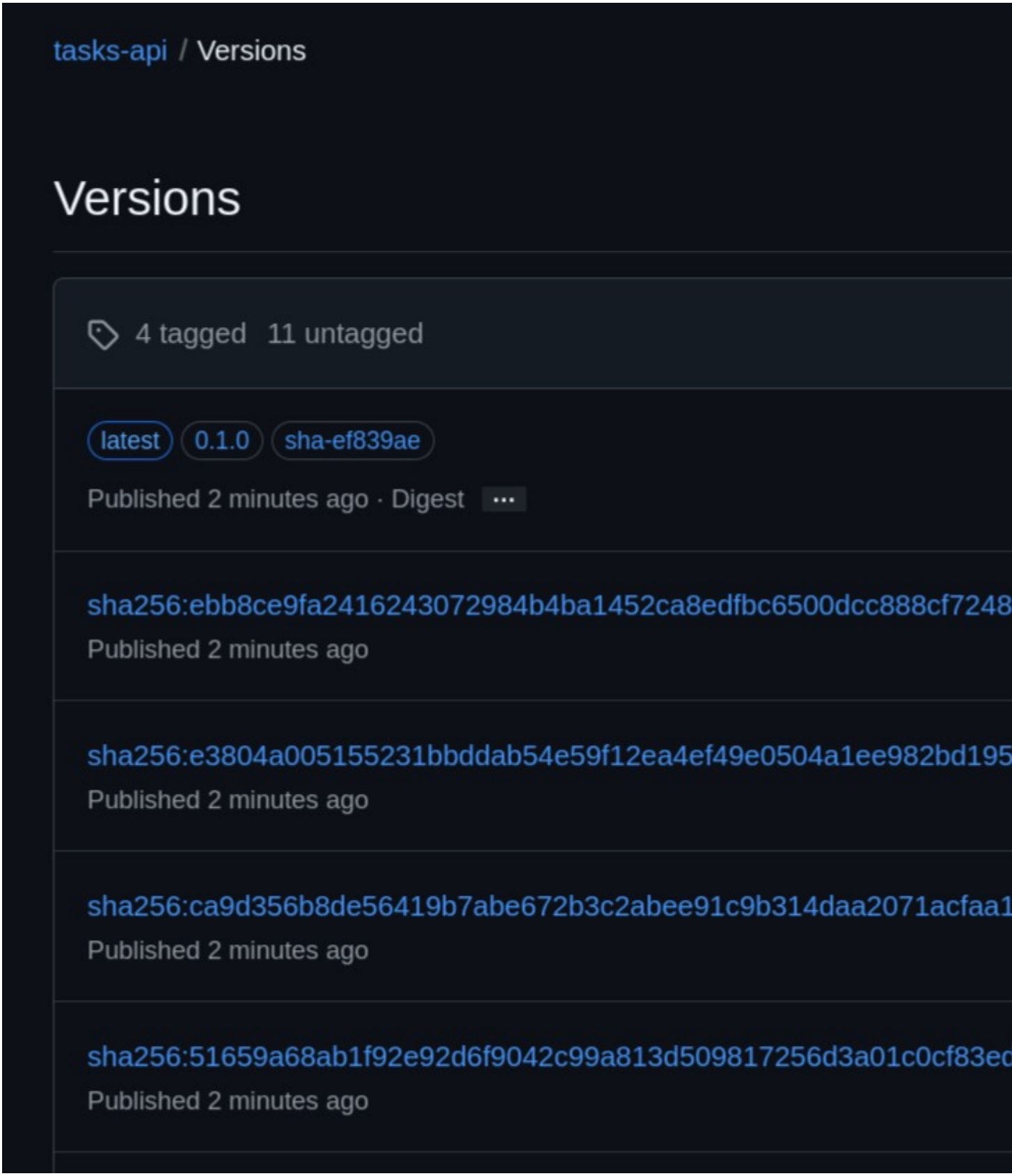
Find this useful? Let us know

ID	Name	Status	Cached	Duration

TP3-4 : La page du package `tasks-api` sur GHCR avec son premier tag `sha-...`.



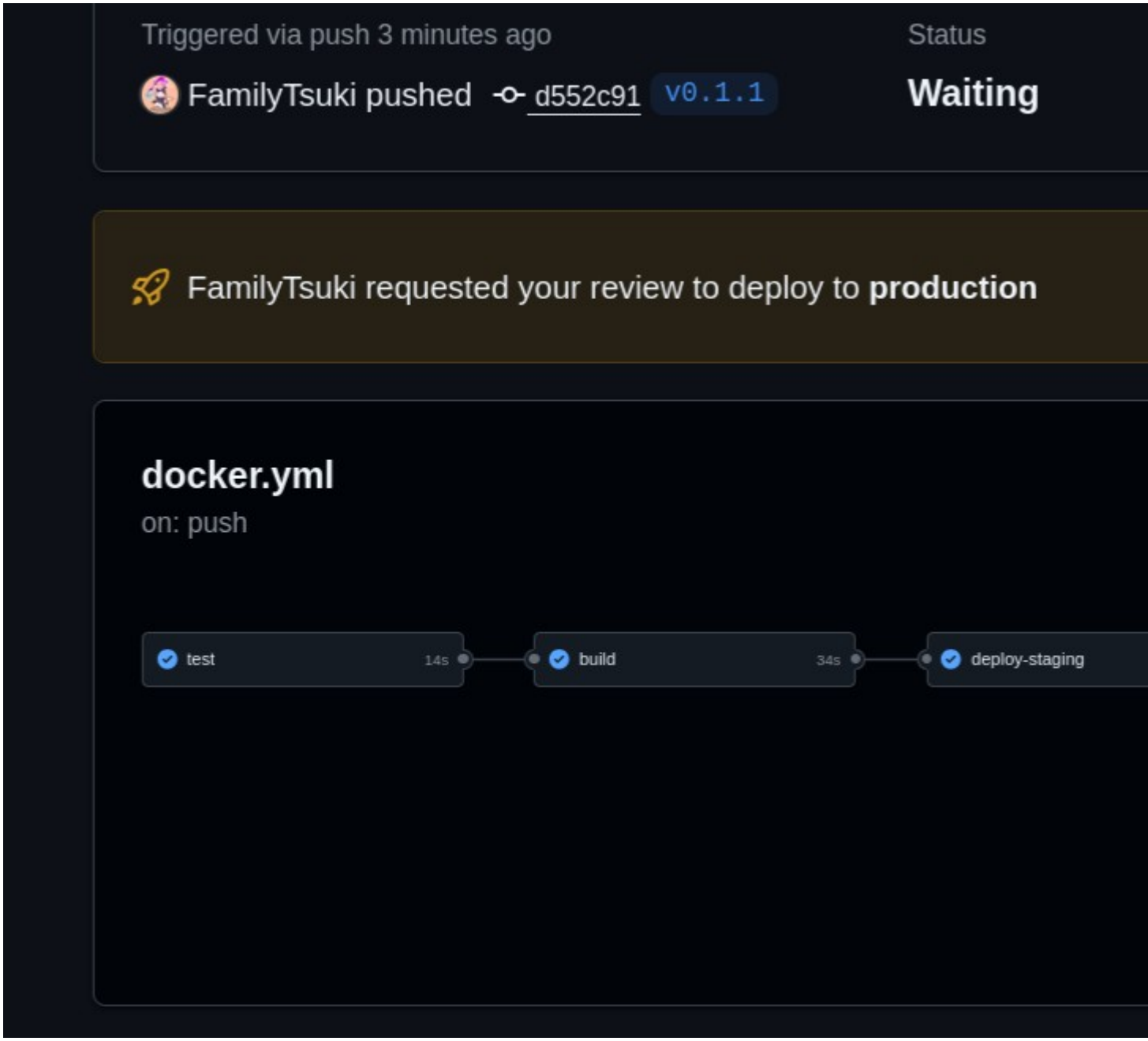
TP3-5 : La liste des tags du package montrant à la fois `0.1.0` et les tags sha.



TP3-6 : `docker run` de l'image téléchargée depuis GHCR et réponse `0.1.0` sur `/version`.

```
● tasks-api tp3 ● ? ❱ curl http://localhost:8000/version
  {"version":"0.1.0"}
```

TP3-7 : Le pipeline en pause en attente d'approbation sur l'environnement production, puis le run terminé.



docker.yml

on: push



build summary

Docker Build summary

For a detailed look at the build, download the following build recor
Docker Desktop's Builds view.
Build records include details such as timing, dependencies, result
information about a build. Learn more

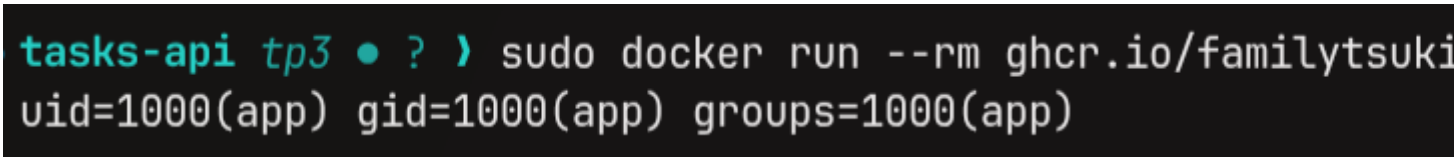
⬇️ **FamilyTsuki~tasks-api~Y69KRI.dockerbuild** (48.05 KB - inc

Find this useful? Let us know

ID	Name	Status	Cached	Duratio
`Y69KRI`	**tasks-api**	✅ completed	0%	10s

▸ **Build inputs**

TP3-8 : Sortie de la commande `id` dans le conteneur confirmant l'utilisateur non-root `uid=1000(app)`.



Réponses aux questions

Q1 : C'est quoi une image ? Un conteneur ? À quoi sert le `.dockerignore` ?

Une image c'est le modèle figé (le code + les dépendances), un conteneur c'est une image qui tourne (on peut lancer plusieurs conteneurs depuis la même image). Le `.dockerignore` sert à pas copier les fichiers inutiles (`.git`, `.venv`, cache...) pour que l'image soit plus légère, que le build soit plus rapide et pour pas mettre de trucs sensibles dedans.

Q2 : Que gagne-t-on à décrire le lancement dans un `docker-compose.yml` plutôt qu'une longue commande ?

Ça évite de retaper toute la ligne avec les ports et les variables à chaque fois, ça accélère le lancement et on est sûr d'avoir la même configuration à chaque fois.

Q3 : Pourquoi `build` attend-il `test` ? Pourquoi éviter le tag `latest` en production ?

Pour pas construire et publier une image avec du code cassé si les tests ratent. `latest` change à chaque push donc on sait jamais quelle version tourne vraiment en prod et c'est dur de revenir en arrière, avec un tag fixe (`0.1.0` ou le sha) on sait exactement ce qui tourne.

Q4 : Quel est l'avantage d'une image versionnée par rapport à « le code de ma machine » ?

L'image c'est toujours la même partout (ma machine, la CI, la prod) avec les mêmes dépendances, alors que le code de ma machine dépend de ce qui est installé localement. Avec une version on sait exactement ce qui tourne et on peut revenir à une ancienne version facilement.

Q5 : À quoi sert un test de fumée ? Pourquoi demander une approbation humaine avant la prod ?

Un test de fumée c'est un test rapide juste après le déploiement (comme un curl sur `/health`) pour vérifier que l'app démarre et répond bien. On demande une validation humaine pour qu'une vraie personne vérifie avant que ça parte en prod vu que c'est utilisé par de vrais utilisateurs, ça évite de déployer un truc pas voulu juste parce que la pipeline est verte.

Q6 : Pourquoi ne pas exécuter une application en root dans un conteneur ? Bonnes pratiques de l'affaire Trivy ?

Ne jamais exécuter en root parce que si quelqu'un trouve une faille dans l'app il a tous les droits dans

le conteneur et ça peut l'aider à s'échapper sur la machine hôte. Avec un utilisateur normal (`USER app`) il peut faire beaucoup moins de dégâts.

Pour Trivy : épingler les actions avec leur commit hash complet plutôt qu'un tag `@v1` modifiable, limiter les permissions du token au minimum par job, et ne jamais exposer de secrets inutiles aux actions tierces.

Difficultés rencontrées

Au début mon curl `/version` répondait `local` au lieu de `0.1.0` parce que mon conteneur Docker Compose de la partie précédente tournait encore sur le port 8000. Il a fallu faire un `docker compose down` pour libérer le port avant de lancer l'image de GHCR.

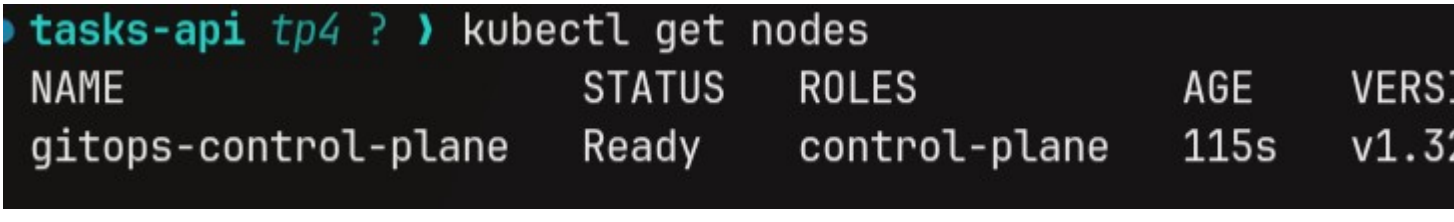
TP4 : GitOps avec Argo CD

Objectif

Dans ce dernier TP on a mis en place du vrai GitOps avec un cluster Kubernetes local (Kind) et Argo CD. On a séparé le dépôt de configuration du dépôt applicatif, observé la correction automatique des dérives (`selfHeal`), et automatisé toute la chaîne : un tag `v0.2.0` sur le code met à jour le manifeste dans le dépôt de config et déclenche le déploiement sans aucune commande manuelle sur le cluster.

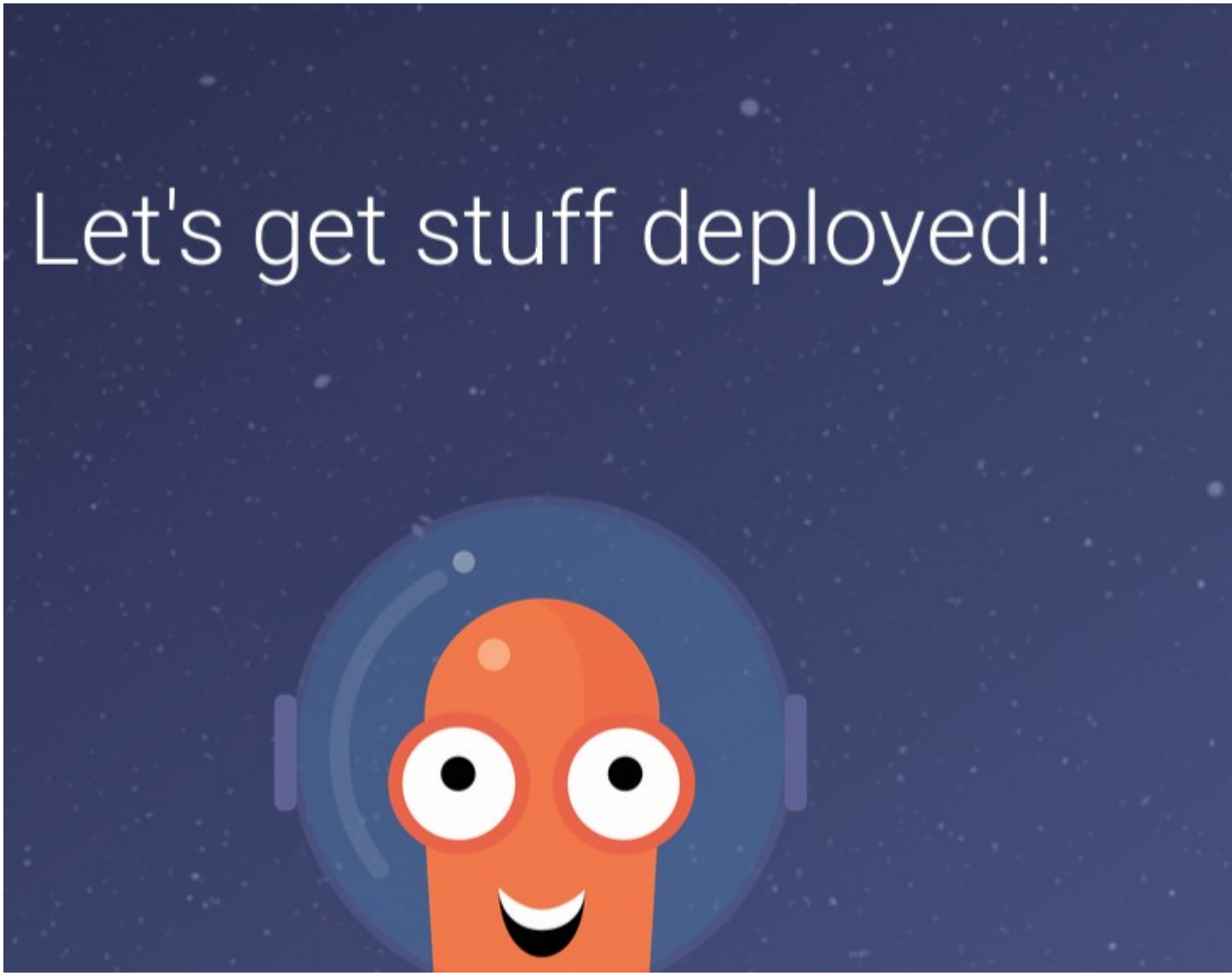
Captures d'écran

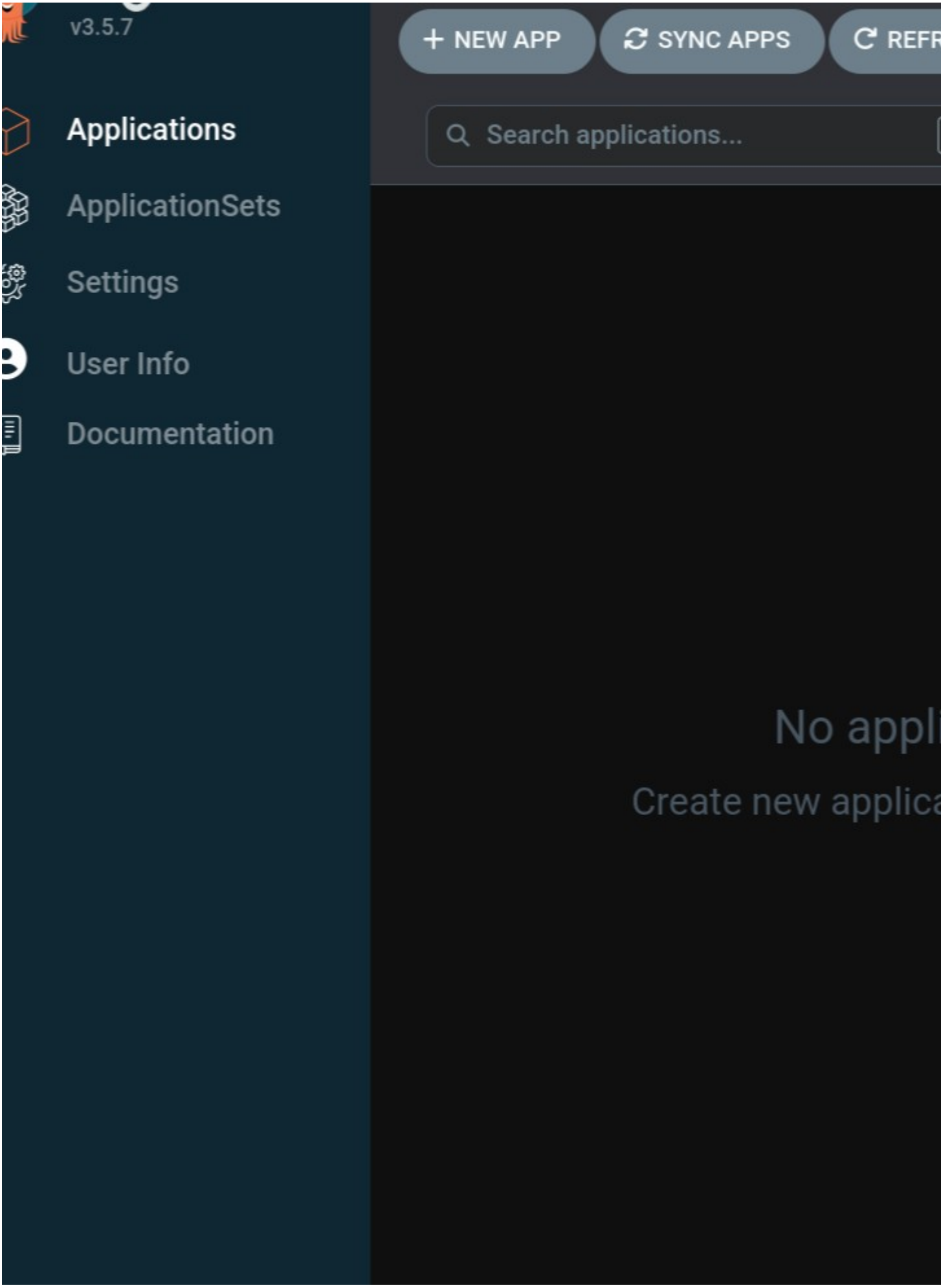
TP4-1 : Le nœud du cluster Kind au statut `Ready` (`kubectl get nodes`).



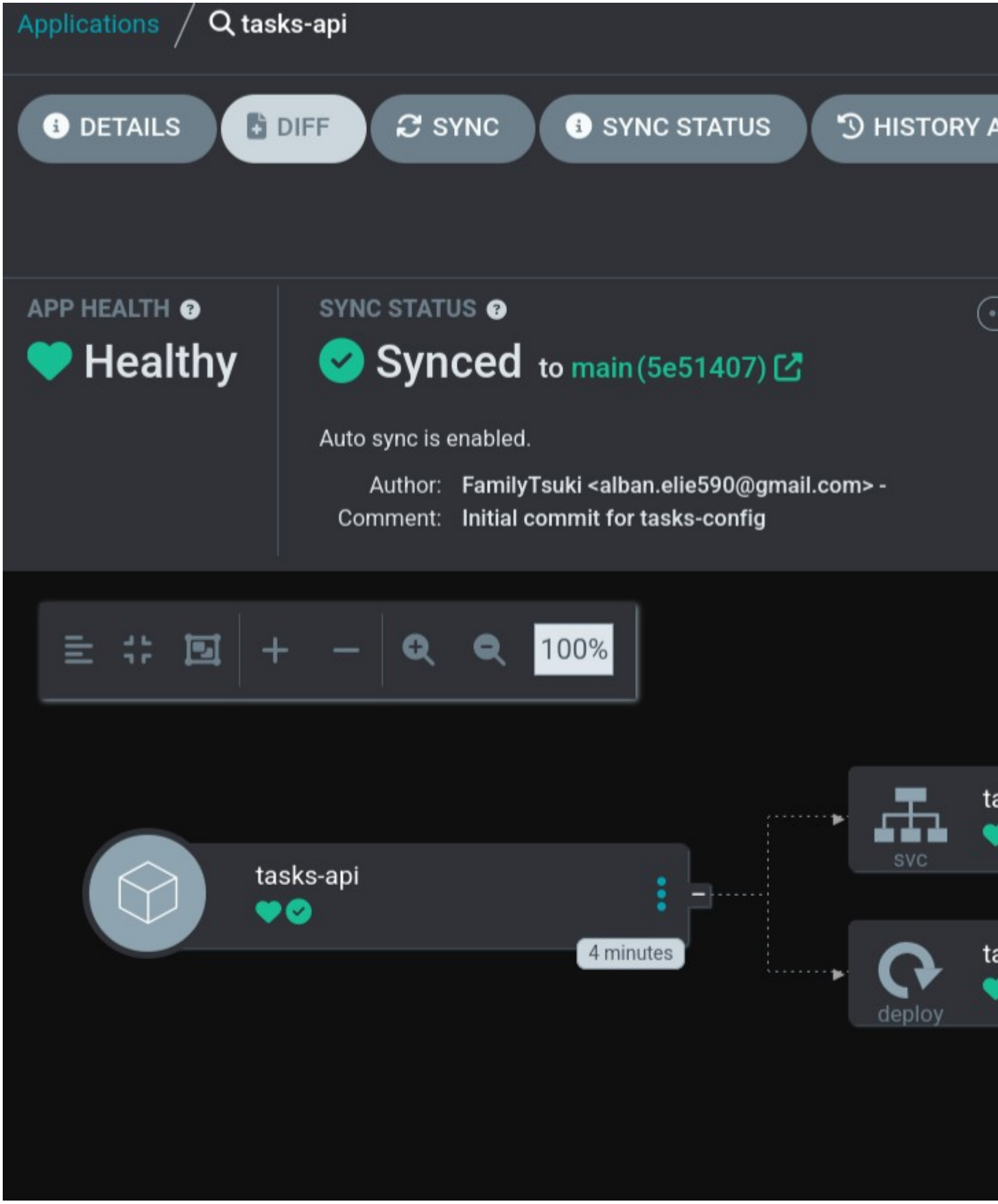
TP4-2 : Tous les pods Argo CD au statut `Running` et la page de connexion web d'Argo CD.

```
tasks-api tp4 ? ❯ kubectl get pods -n argocd
NAME                                                READY
argocd-application-controller-0                     1/1
argocd-applicationset-controller-6848dcccb-tpd4t    1/1
argocd-dex-server-6779f5df87-szjbp                  1/1
argocd-notifications-controller-75cc4fb684-76c5p    1/1
argocd-redis-75966c957d-pr8sn                       1/1
argocd-repo-server-7546f8f546-zqql9                 1/1
argocd-server-8df48b7cc-9nmwk                       1/1
```




v3.5.7
Applications
ApplicationSets
Settings
User Info
Documentation
+ NEW APP
SYNC APPS
Search applications...
No appli
Create new applica

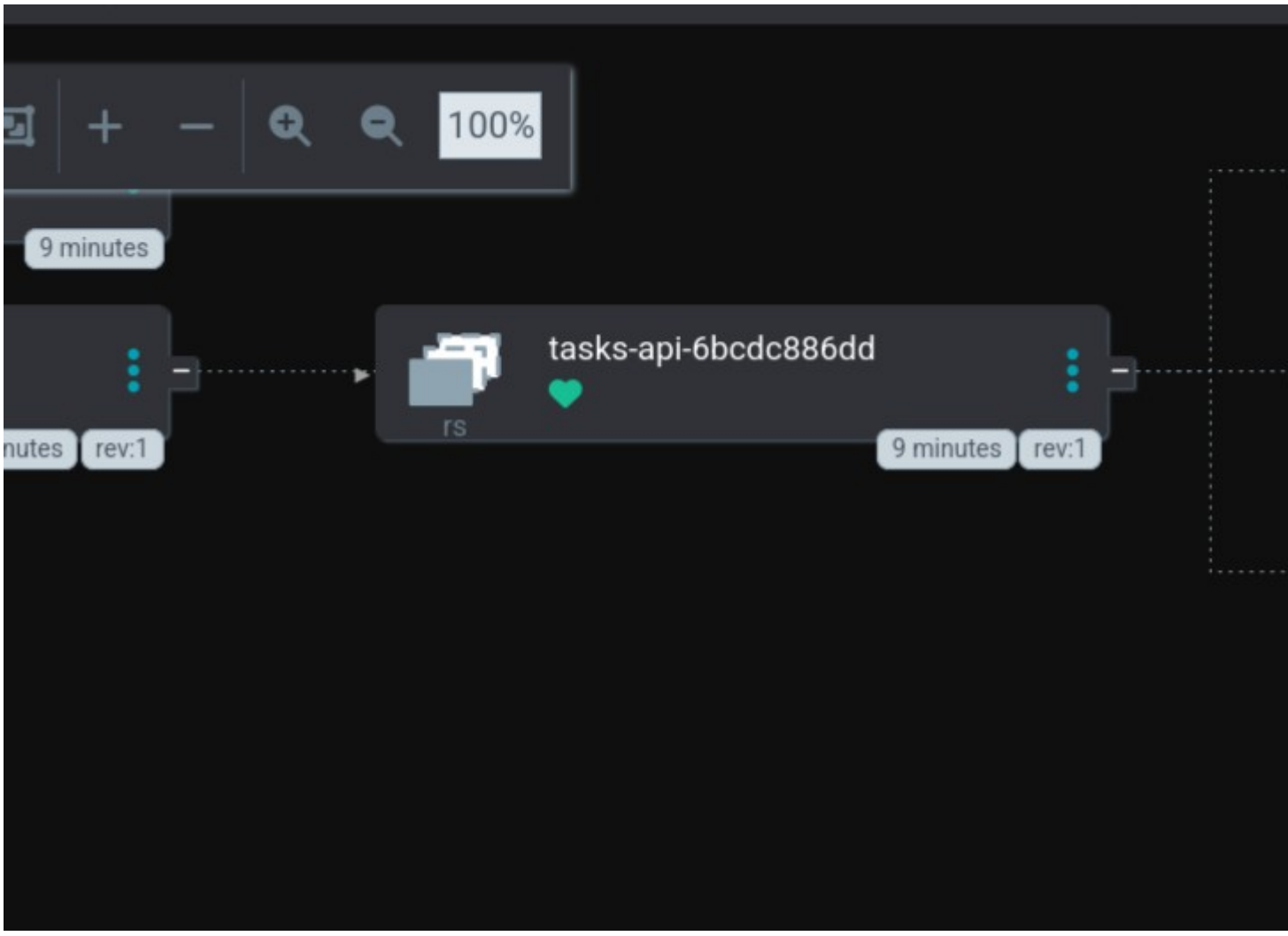
TP4-3 : L'application `tasks-api` dans Argo CD avec le statut `Synced` et `Healthy` et ses 2 pods.



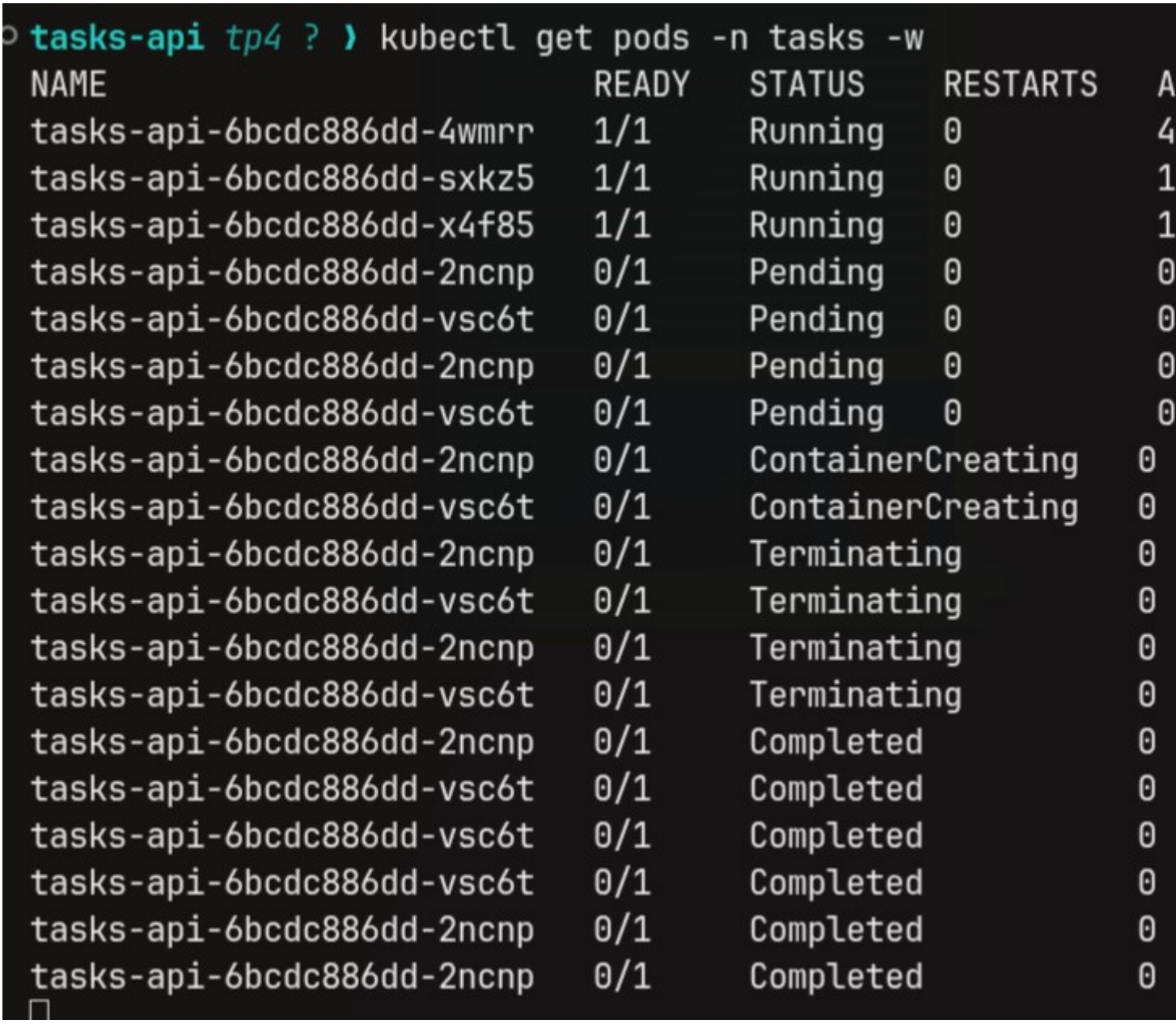
TP4-4 : La commande `curl` sur le port 8081 qui répond la version `0.1.0` déployée dans le cluster.

```
tasks-api tp4 ? ❯ curl http://localhost:8081/version
{"version":"0.1.0"}
```

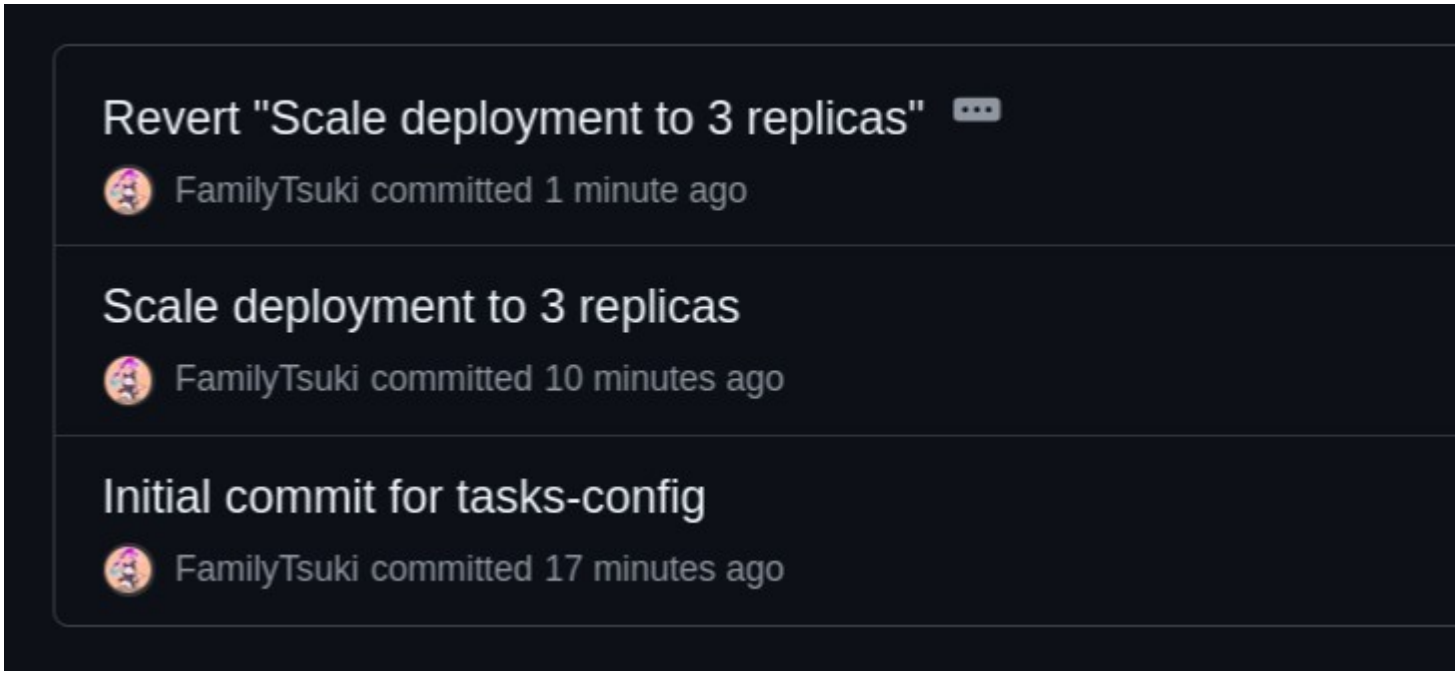
TP4-5 : Le commit passant à `replicas: 3` dans `tasks-config` et les 3 pods visibles dans Argo CD.

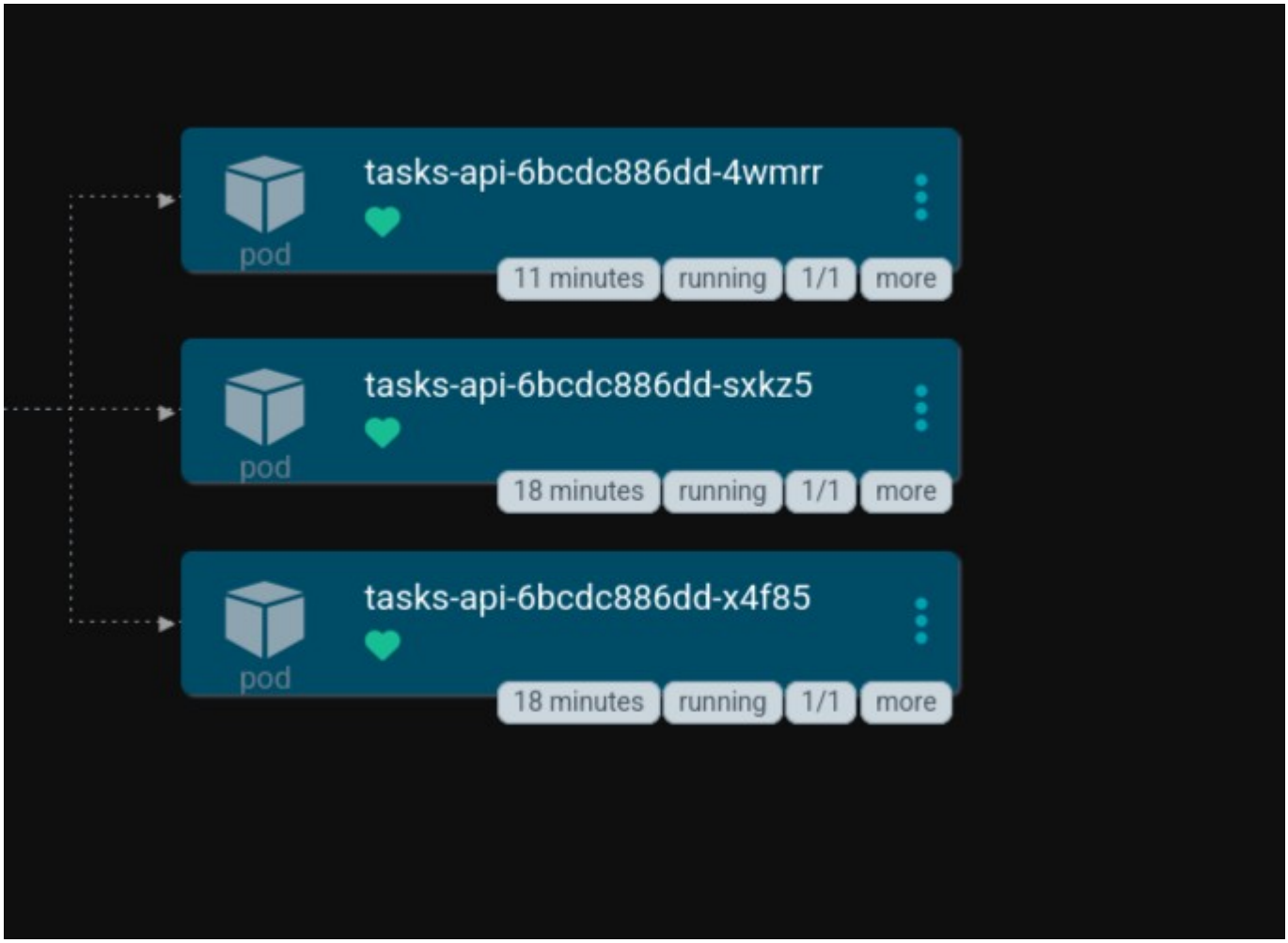


TP4-6 : La commande `kubectl scale` à 5 pods et Argo CD qui supprime direct les pods en trop (`selfHeal`).



TP4-7 : L'historique Git avec le `revert` dans `tasks-config` et le retour automatique à 2 pods.

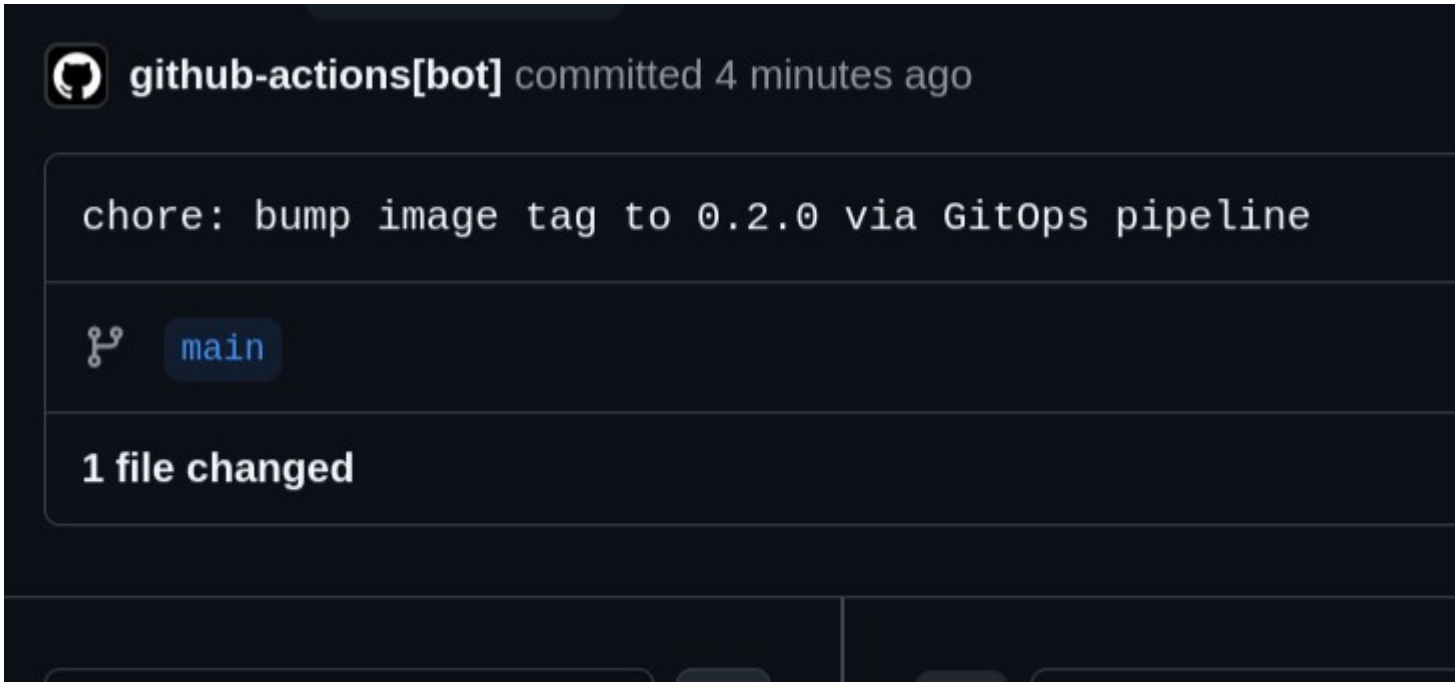


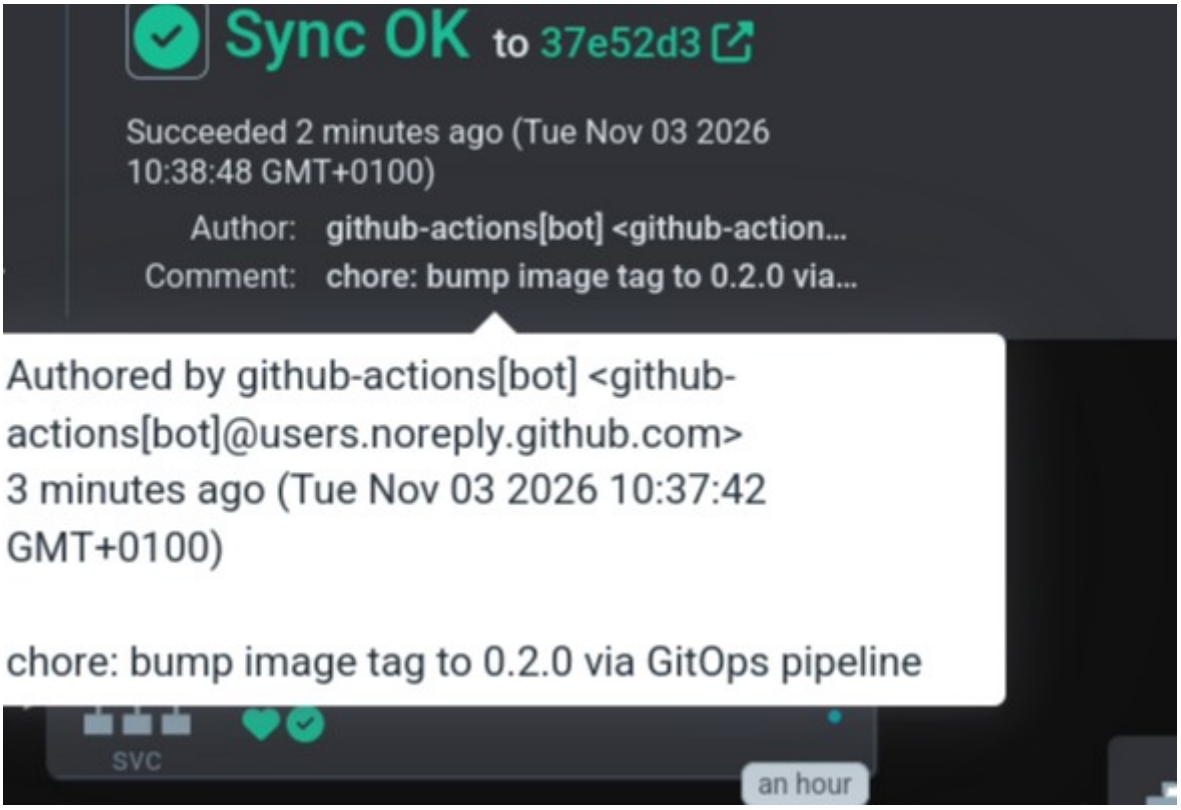


TP4-8 : Le workflow `docker.yml` terminé avec tous les jobs au vert, y compris `update-config`.



TP4-9 : Le commit automatique de `github-actions[bot]` dans `tasks-config` et Argo CD synchronisé sur `0.2.0`.





TP4-10 : La commande `curl` sur l'API qui répond bien la version `0.2.0` mise à jour.

```
tasks-api main ? × curl http://localhost:8081/version
{"version":"0.2.0"}
```

Réponses aux questions

Q1 : C'est quoi un pod ? Un deployment ? Pourquoi un namespace ?

- *Pod :* plus petite unité dans Kubernetes, un ou plusieurs conteneurs qui partagent le même réseau et les mêmes volumes.

- *Deployment :* objet qui gère les pods et leurs réplicats, gère les rolling updates sans coupure et les retours en arrière.

- *Namespace :* isolation logique dans le cluster pour séparer les projets ou environnements et éviter les conflits de nom.

Q2 : Pourquoi sépare-t-on le dépôt de l'application et le dépôt de configuration ?

Pour éviter que la CI tourne en boucle quand elle modifie les manifestes, pour bien séparer les droits des devs (code) et des ops (infrastructures), et pour garder un historique propre et traçable sur chaque dépôt.

Q3 : Que signifie `selfHeal` ? Pourquoi le retour arrière est-il simple en GitOps ?

`selfHeal` veut dire qu'Argo CD compare en permanence le cluster avec le dépôt Git : si quelqu'un modifie un truc à la main sur le cluster (par exemple un scale à 5), Argo CD détecte la dérive et remet automatiquement ce qui est écrit dans Git (3 réplicas).

Le retour arrière est simple parce que chaque déploiement correspond à un commit dans `tasks-config` : il suffit de faire un `git revert` et de pousser, Argo CD applique l'ancienne version tout seul sans qu'on ait besoin de toucher au cluster.

Q4 : Résumez en 5 lignes le chemin d'un tag Git jusqu'au pod. Qui pousse et qui tire ? Différence avec un déploiement classique ?

1. Je pousse le tag `v0.2.0` sur le dépôt applicatif, ce qui déclenche la pipeline GitHub Actions.

2. La CI teste le code, construit l'image Docker et la publie sur GHCR avec le tag `0.2.0`.

3. Après le test staging et la validation de la prod, le job `update-config` met à jour `deployment.yaml` dans `tasks-config` et commite.

4. Argo CD surveille `tasks-config`, voit que le dépôt a changé et que le cluster n'est plus synchronisé.

5. Argo CD applique le manifeste et Kubernetes remplace les anciens pods par les nouveaux avec l'image `0.2.0`.

Qui pousse / qui tire : La CI pousse l'image sur GHCR et le commit dans `tasks-config`, mais ne touche jamais au cluster. C'est Argo CD qui tire la config depuis Git, et Kubernetes qui tire l'image depuis GHCR.

Différence avec la CI/CD classique : En classique, c'est la pipeline qui a les clés du cluster et fait un `kubectl apply` ou un SSH en direct, donc si quelqu'un modifie le cluster à la main personne ne le voit. En GitOps, le cluster n'est pas ouvert à la CI, Git est la seule source de vérité et les dérives sont corrigées automatiquement.

Difficultés rencontrées

Pour le test du `selfHeal`, Argo CD a corrigé la dérive tellement vite (moins d'une seconde) que la première fois les pods étaient déjà supprimés quand j'ai lancé `kubectl get pods`. Il a fallu lancer la commande avec `-w` dans un premier terminal avant de faire le `kubectl scale` dans un second terminal pour bien capturer l'événement en direct.

Bilan personnel

Ce cours m'a permis de comprendre concrètement tout le cycle d'un projet moderne, de l'écriture du code jusqu'au cluster de production. Ce que j'ai surtout retenu, c'est la puissance de GitHub Actions pour automatiser les tests et la sécurité des images, et le principe du GitOps avec Argo CD où Git devient la seule source de vérité pour le déploiement. Ce qui m'a posé le plus de problèmes au début, c'était la gestion fine des permissions (jetons PAT, droits de workflow) et les pièges bêtes comme l'indentation YAML ou les formats de tags. Si c'était à refaire, je ferais plus attention à bien isoler mes branches de test dès le départ et je penserais à lancer mes surveillances terminal (`-w`) avant d'exécuter des modifications sur Kubernetes.